Uniwersytet Warmińsko-Mazurski w Olsztynie
Wydział Matematyki i Informatyki

Kierunek: Informatyka

Maciej Kała

Interaktywny, Efektowny Program Eksplorujący Fraktale

Praca inżynierska wykonana
w Katedrze Metod Matematycznych Informatyki
pod kierunkiem
dr inż. Bartosza Nowaka

Olsztyn, 2026 rok

1:9843694870

University of Warmia and Mazury in Olsztyn
Faculty of Mathematics and Computer Science

Field of Study: Computer Science

Maciej Kała

Interactive, Impressive Fractal Exploration Program

Engineer's Thesis is performed
in Department of Mathematical Methods
in Computer Science (Informatics)
under supervision of
Ph.D. Bartosz Nowak

Olsztyn, 2026

2:5794866631

Spis treści

Wstęp . . . 4
Motywacja . . . 4
Opis problemu . . . 4
Przegląd istniejących rozwiązań . . . 5
Cel i zakres pracy . . . 5
Rozdział 1. Wstęp teoretyczny . . . 8
1.1. Samopodobieństwo i wieloskalowość . . . 8
1.2. Wymiar fraktalny . . . 9
1.3. Rys historyczny . . . 9
1.4. Podstawy matematyczne . . . 9
1.4.1. Liczby zespolone . . . 10
1.4.2. Iteracja i orbity . . . 10
1.4.3. Zbiory Julii i zbiór Mandelbrota . . . 10
1.5. Klasyfikacja fraktali . . . 11
Rozdział 2. Algorytmy generowania fraktali . . . 12
2.1. Ogólny opis algorytmu fraktalnego . . . 12
Odwzorowanie piksela na płaszczyznę zespoloną . . . 12
2.2. Fraktale escape-time a fraktale punktowe . . . 13
2.3. Pseudokod algorytmów i omówienie kroków obliczeń . . . 13
2.3.1. Zbiór Mandelbrota . . . 13
2.3.2. Zbiór Julii . . . 14
2.3.3. Burning Ship . . . 14
2.3.4. Wygładzanie kolorowania iteracyjnego . . . 15
2.3.5. Trójkąt Sierpińskiego — chaos game . . . 16
2.3.6. Paproć Barnsleya — iterowany układ funkcji (IFS) . . . 17
Rasteryzacja chmury punktów . . . 17
2.4. Parametryzacja i wpływ parametrów na wygląd fraktala . . . 18
2.5. Złożoność obliczeniowa i czynniki wpływające na czas renderowania . . . 18
2.6. Przykłady efektów wizualnych i powiązania między fraktalami . . . 19
Rozdział 3. Założenia projektowe i wymagania aplikacji . . . 20
3.1. Scenariusze użycia i założenia funkcjonalne . . . 20
3.2. Wymagania i priorytety . . . 20
Wymagania funkcjonalne . . . 21
Wymagania niefunkcjonalne . . . 21
3.3. Założenia procesu tworzenia projektu . . . 23
3.4. Docelowe środowisko użytkownika . . . 23
Rozdział 4. Implementacja aplikacji . . . 24
4.1. Wybór technologii . . . 24
4.2. Architektura systemu . . . 25

3:4772817799

4.3. Separacja warstwy logicznej i interfejsu . . . 25
Klasa bazowa FractalAlgorithm . . . 25
Klasa ReactiveParams . . . 25
Enum AlgoChoice . . . 26
4.4. Mechanizm komunikacji i przepływ zdarzeń . . . 26
4.5. System wątków demonicznych . . . 26
4.5.1. RedrawDaemon — odświeżanie obrazu . . . 26
4.5.2. Resize i zarządzanie teksturą . . . 27
4.5.3. KeyboardDaemon — obsługa klawiatury i ruchu . . . 27
4.6. Struktura kodu i klasy bazowe . . . 27
Typy współdzielone (common/types.py) . . . 27
Adaptacyjne obniżanie jakości (UIState.generate_frame) . . . 28
4.7. Zarządzanie zależnościami . . . 28

Rozdział 5. Projekt interfejsu użytkownika . . . 29
5.1. Założenia UX/UI . . . 29
5.2. Organizacja paneli . . . 29
Panel obrazu (TexturePanel) . . . 29
Panel boczny . . . 29
5.3. Sterowanie kamerą . . . 30
5.4. Funkcje użytkowe . . . 30

Rozdział 6. Kolorowanie fraktali . . . 32
6.1. Algorytmy kolorowania . . . 32
Normalizacja i LUT . . . 32
Interpolacja gradientu . . . 32
Korekcja gamma . . . 32
6.2. Presety i kolorowanie niestandardowe . . . 33
Tryb presetów . . . 33
Tryb niestandardowy . . . 33

Rozdział 7. Animacja i eksport . . . 36
7.1. Animacja parametrów, klatki, przebieg animacji . . . 36
7.2. Interpolacja ruchu . . . 36
7.3. Format i opcje eksportu . . . 37

Rozdział 8. Wyniki i analiza działania programu . . . 38
8.1. Poprawność generacji fraktali . . . 38
8.2. Ciekawe obszary i obserwacje wizualne . . . 39
8.2.1. Zbiór Mandelbrota . . . 40
8.2.2. Zbiory Julii . . . 41
8.2.3. Burning Ship . . . 43
8.3. Wydajność, czas renderowania, testy limitów . . . 43
8.3.1. Środowisko testowe . . . 43
8.3.2. Metodologia pomiarów . . . 44
8.3.3. Odchylenie standardowe . . . 44
8.3.4. Wyniki dla fraktali escape-time . . . 44
8.3.5. Wyniki dla fraktali IFS . . . 46
8.3.6. Koszt kompilacji JIT . . . 46
8.3.7. Wnioski wydajnościowe . . . 47
8.4. Ograniczenia przybliżania i kompromisy jakości . . . 47
8.4.1. Ograniczenie precyzji arytmetyki `float64` . . . 47
8.4.2. Adaptacyjne obniżanie jakości podczas nawigacji . . . 48

4:1269704496

Rozdział 9. Porównanie z istniejącymi rozwiązaniami . . . 50
9.1. Przegląd programów dostępnych w internecie . . . 50
9.2. Różnice funkcjonalne i jakościowe . . . 50
9.3. Wnioski z porównania . . . 51
Rozdział 10. Dyskusja . . . 52
10.1. Wnioski poznawcze . . . 52
10.2. Wpływ kolorowania na percepcję i odbiór fraktali . . . 52
10.3. Ograniczenia przyjętego podejścia . . . 52
10.4. Potencjalne zastosowania aplikacji . . . 53
Rozdział 11. Kierunki rozwoju . . . 54
11.1. Dodatkowe fraktale . . . 54
11.2. Fraktale trójwymiarowe . . . 54
11.3. Usprawnienia wydajności . . . 54
11.4. Wariant edukacyjny . . . 55
Rozdział 12. Podsumowanie . . . 56
12.1. Zrealizowane funkcje . . . 56
12.2. Rezultaty i wnioski . . . 57
12.3. Rozwój umiejętności i doświadczeń . . . 57
Słownik . . . 59
Bibliografia . . . 62
Dodatek . . . 63
12.4. Wymagania i instalacja . . . 63
12.5. Uruchomienie programu . . . 63
12.6. Opis funkcji i sterowania . . . 64
12.6.1. Eksport i przykładowy workflow . . . 65
Podziękowanie . . . 66
Streszczenie . . . 67
Abstract . . . 68

5:5491608347

Wstęp

Fraktale stanowią szczególną klasę obiektów matematycznych, charakteryzujących się złożoną strukturą na każdym poziomie skali. Cecha ta sprawia, że fraktale są interesujące zarówno poznawczo, jak i wizualnie, będąc przykładem jak prosta reguła obliczeniowa może prowadzić do form o wysokiej złożoności i rozpoznawalnych motywach.

Zjawiska o fraktalnym charakterze są obserwowalne w przyrodzie w postaci rozgałęzień roślin, struktur sieci rzecznych, ukształtowania linii brzegowych, oraz procesów wzrostu i erozji. Z tego względu fraktale bywają traktowane jako narzędzie przybliżonego opisu form nieregularnych, a nie jedynie jako abstrakcyjna konstrukcja. Równocześnie fraktale odegrały istotną rolę w rozwoju grafiki komputerowej i wizualizacji naukowej, pozwalając na syntetyczne generowanie obrazów o dużej różnorodności kształtów, w tym takich, które przypominają struktury naturalne. Temat fraktali łączy kilka dziedzin: matematykę — poprzez definicje i twierdzenia, informatykę — obliczenia numeryczne i optymalizację, a także aspekt estetyczny i artystyczny związany z projektowaniem obrazu, a w szczególności doborem kolorowania i sposobem prezentacji.

Motywacja

Motywacją pracy jest opracowanie narzędzia umożliwiającego interaktywną eksplorację fraktali, rozumianą jako swobodne poruszanie się po obrazie, zmianę skali obserwacji oraz modyfikację parametrów wpływających na kształt generowanej struktury. W przeciwieństwie do statycznych wizualizacji, interaktywne podejście pozwala obserwować zależności pomiędzy definicją matematyczną a efektem wizualnym w sposób bezpośredni i intuicyjny. Dzięki temu użytkownik może analizować fraktal jako obiekt matematyczny lub też jako strukturę graficzną, przeprowadzając „eksperymenty” poprzez kontrolę parametrów i obserwację rezultatów w czasie rzeczywistym.

Taka wizualizacja fraktali wymaga pogodzenia sprzecznych oczekiwań: uzyskania obrazu o wysokiej jakości przy jednoczesnym zachowaniu płynności sterowania. W praktyce prowadzi to do konieczności projektowania rozwiązań, które zapewniają responsywny interfejs użytkownika pomimo kosztownych obliczeń. Z tego względu praca koncentruje się nie tylko na samych algorytmach generowania fraktali, lecz także na zaprojektowaniu sposobu interakcji i prezentacji wyników w formie użytecznej dla użytkownika.

Opis problemu

Generowanie fraktali wiąże się z wysokimi wymaganiami obliczeniowymi: wyznaczenie jednej klatki obrazu o typowej rozdzielczości ekranu wymaga wykonania dziesiątek milionów operacji zmiennoprzecinkowych. Każdy piksel jest niezależny od pozostałych, jednak łączny koszt

6:1550092078

obliczeń rośnie proporcjonalnie do iloczynu rozdzielczości i liczby iteracji. Jednocześnie aplikacja interaktywna musi reagować na polecenia użytkownika bez odczuwalnego opóźnienia — zatrzymanie obrazu w trakcie nawigacji lub zmiany parametrów byłoby sprzeczne z założeniem eksploracji w czasie rzeczywistym. Pogodzenie tych dwóch wymagań jest jednym z centralnych problemów implementacyjnych pracy.

Następną trudnością jest matematyczna różnorodność fraktali: algorytmy escape-time (zbiór Mandelbrota, zbiory Julii, Burning Ship) i algorytmy oparte na iterowanych układach funkcji (trójkąt Sierpińskiego, paproć Barnsleya) różnią się zasadniczo strukturą obliczeń i sposobem uzyskania obrazu wynikowego. Fraktale escape-time przypisują wartość każdemu pikselowi siatki niezależnie, natomiast fraktale punktowe generują chmurę punktów w przestrzeni ciągłej, która musi zostać rasteryzowana na dyskretną siatkę pikseli. Opracowanie jednolitego potoku renderowania obsługującego obie klasy algorytmów wymagało zaprojektowania odpowiedniej warstwy abstrakcji.

Przegląd istniejących rozwiązań

Interaktywna eksploracja fraktali jest tematem podejmowanym przez wiele projektów programistycznych. Istniejące rozwiązania można podzielić na trzy główne kategorie. Pierwszą stanowią narzędzia ogólnego przeznaczenia, takie jak XaoS — otwartoźródłowy program oferujący szeroki zestaw fraktali i bardzo wydajny tryb iteratywnego renderowania, lecz pozbawiony modułu animacji opartego o klatki kluczowe. Drugą kategorią są profesjonalne pakiety komercyjne, takie jak Ultra Fractal, oferujące rozbudowany system warstw i własny język formuł, lecz wymagające znaczącej wiedzy. Trzecią — wąsko wyspecjalizowane narzędzia, takie jak Kalles Fraktaler, skupione na pojedynczym typie fraktala (zbiór Mandelbrota) i głębokim przybliżeniu z wykorzystaniem arytmetyki arbitralnej precyzji.

Niniejsza praca lokuje się pomiędzy tymi kategoriami: zakresem fraktali zbliża się do XaoS, lecz oferuje dodatkowo moduł animacji z eksportem do popularnych formatów wideo, a od narzędzi profesjonalnych odróżnia ją prosty, edukacyjny charakter i otwarta architektura kodu. Szczegółowe porównanie zaimplementowanej aplikacji z wybranymi programami zewnętrznymi przedstawiono w rozdziale 9.

Cel i zakres pracy

Celem pracy jest zaprojektowanie i zaimplementowanie aplikacji desktopowej umożliwiającej interaktywną eksplorację fraktali, ze szczególnym uwzględnieniem płynności nawigacji i jakości wizualizacji przy zachowaniu przejrzystej architektury kodu. Zakres funkcjonalny obejmuje:

— implementację pięciu algorytmów fraktalnych: zbioru Mandelbrota, zbioru Julii, Burning Ship, trójkąta Sierpińskiego i paproci Barnsleya,
— nawigację kamerą za pomocą klawiatury (przesunięcie i zoom) z mechanizmem adaptacyjnego obniżania jakości renderowania podczas ruchu,
— system reaktywnych parametrów z natychmiastowym odzwierciedleniem zmian w renderowanym obrazie,
— konfigurowalne kolorowanie: presety barwne i tryb niestandardowy ze swobodnie definiowanymi punktami kontrolnymi gradientu,
— moduł animacji oparty na klatkach kluczowych z interpolacją liniową i krzywą Catmull-Rom,

7:8363489798

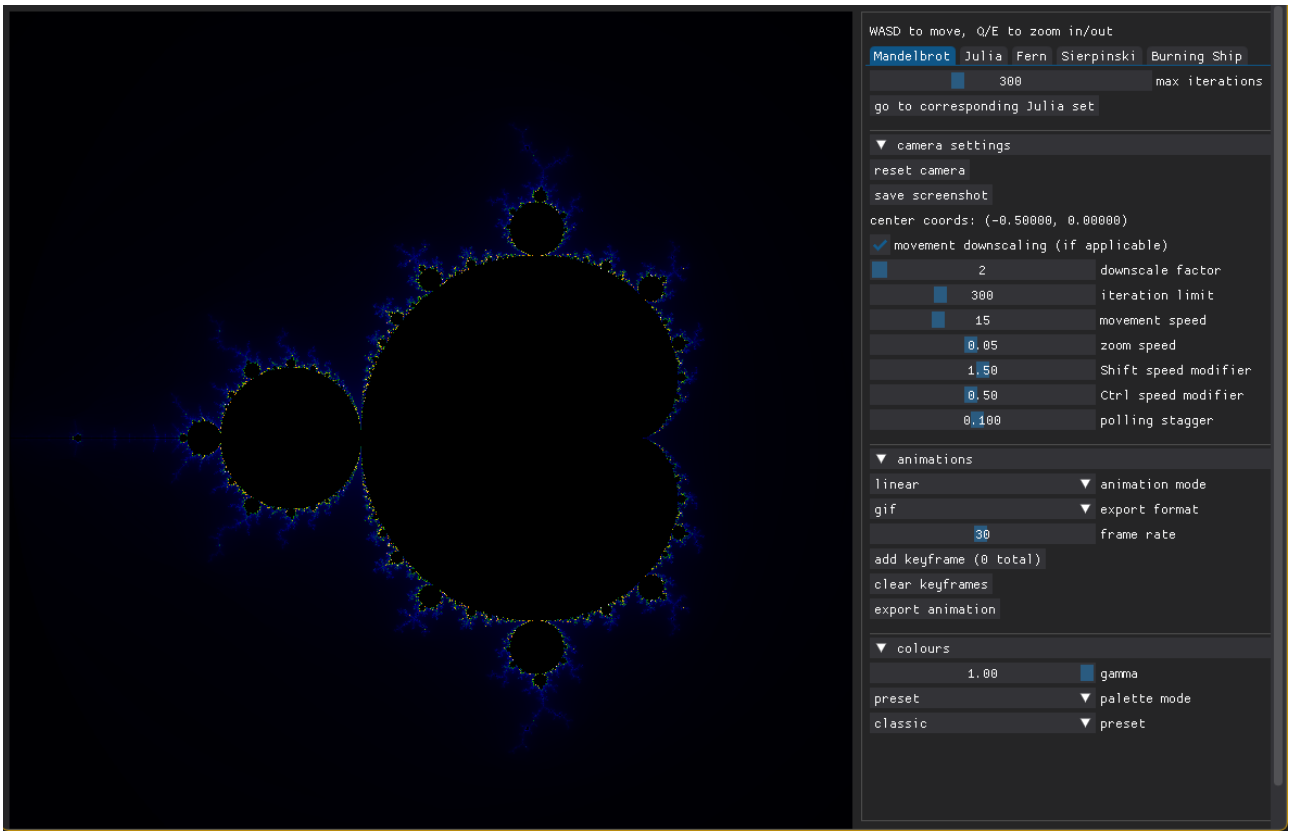
— eksport wyników do formatów PNG (zrzut ekranu), GIF, MP4 i WebM.

Zakres pracy obejmuje również analizę wybranych algorytmów pod kątem złożoności obliczeniowej, omówienie zastosowanych technik optymalizacyjnych (kompilacja JIT, memoizacja, adaptacyjne skalowanie) oraz porównanie zrealizowanego rozwiązania z wybranymi programami dostępnymi publicznie.

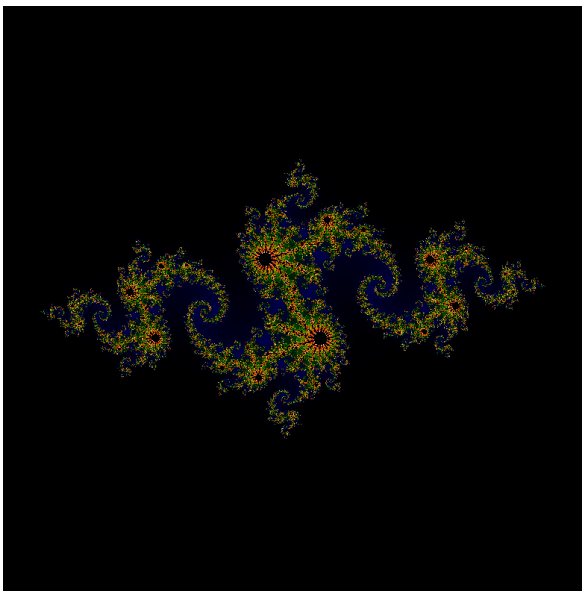
Zaimplementowane algorytmy podzielone są na dwie kategorie opisane szczegółowo w rozdziale 2. Fraktale escape-time — zbiór Mandelbrota, zbiory Julii i Burning Ship — generowane są przez badanie zbieżności iteracji dla każdego piksela. Fraktale punktowe — trójkąt Sierpińskiego i paproć Barnsleya — generowane są metodą losowych przekształceń afinicznych i rasteryzowane na siatkę pikseli.

Aplikacja umożliwia swobodne przybliżanie i oddalanie widoku, eksplorację dowolnych obszarów fraktala, interaktywną zmianę parametrów algorytmu, konfigurację kolorowania oraz nagrywanie animacji z eksportem do popularnych formatów wideo. Szczegółowy opis wszystkich funkcji przedstawiony jest w kolejnych rozdziałach pracy.

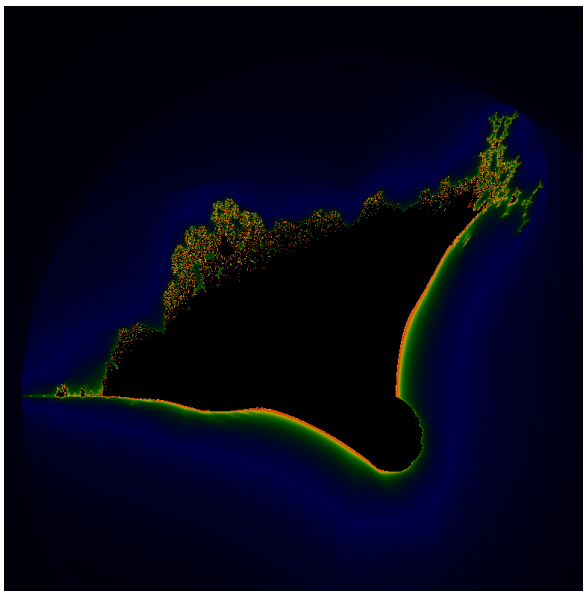
Rysunek 1 przedstawia główne okno opracowanej aplikacji wraz z aktywnym widokiem fraktala Mandelbrota, a pozostałe fraktale przedstawione są na rysunku 2.



Rysunek 1: Główne okno opracowanej aplikacji; szczegółowy opis interfejsu znajduje się w rozdziale 5.



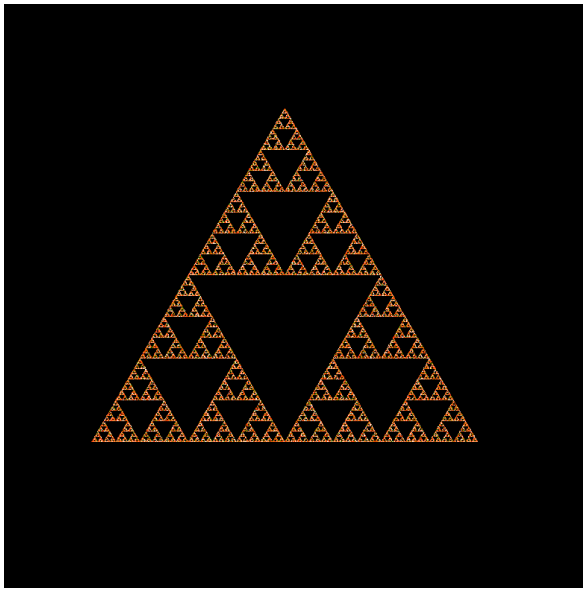
(a) Fraktal Julii.



(b) Fraktal Burning Ship.



(c) Paproć Barnsleya.



(d) Trójkąt Sierpińskiego.

Rysunek 2: Przedstawienie graficzne pozostałych fraktali programu FractalSurfer.

9:1078155794

Rozdział 1

Wstęp teoretyczny

W rozdziale przedstawiono podstawy pojęciowe i matematyczne niezbędne do zrozumienia, w jaki sposób fraktale są definiowane, klasyfikowane i generowane w systemach komputerowych. Fraktale stanowią szczególną klasę obiektów wymykających się intuicjom wypracowanym na gruncie geometrii euklidesowej. Zamiast gładkich krzywych i brył o skończonej złożoności otrzymuje się zbiory o bogatej strukturze, często o poszarpanych granicach, a jednocześnie wykazujące uporządkowanie wynikające z prostej reguły konstrukcyjnej [2, rozdz. 2].

W kontekście implementacji programistycznej szczególnego znaczenia nabierają fraktale generowane metodami iteracyjnymi, ponieważ naturalnie przekładają się one na algorytm „piksel po pikselu": dla każdego punktu płaszczyzny (najczęściej utożsamianego z liczbą zespoloną) bada się przebieg iteracji oraz podejmuje decyzję o przynależności do zbioru bądź o „ucieczce" iteracji do nieskończoności [7, rozdz. 13.4]. W najprostszym ujęciu rdzeniem wielu takich konstrukcji jest iterowany układ postaci:

$$z_{n+1} = f(z_n,\, c) \tag{1.1}$$

gdzie z_n jest stanem iteracji w kroku n, c pełni rolę parametru (często również liczby zespolonej), a funkcja f definiuje regułę przejścia. W zależności od tego, czy zmiennym argumentem jest c czy punkt startowy z_0, otrzymuje się różne interpretacje i różne klasy zbiorów.

W dalszej części rozdziału omówione zostają kolejno: pojęcie samopodobieństwa i wieloskalowości, wymiar fraktalny jako narzędzie opisu złożoności struktury, zarys historyczny, fundamenty matematyczne niezbędne do implementacji fraktali iteracyjnych oraz klasyfikacja fraktali istotna z perspektywy praktycznej.

1.1. Samopodobieństwo i wieloskalowość

Jedną z najczęściej przywoływanych własności fraktali jest *samopodobieństwo* — własność polegająca na tym, że fragment obiektu, po odpowiednim przeskalowaniu, jest nieodróżnialny od całości lub od innych jego fragmentów [2, rozdz. 6]. W klasycznych przykładach geometrycznych (trójkąt Sierpińskiego, płatek Kocha) samopodobieństwo ma charakter ścisły: każdy podobiekt jest dokładną kopią całości pomniejszoną w ustalonym stosunku. W zjawiskach naturalnych — takich jak linie brzegowe, struktury chmur czy rozgałęzienia roślin — mamy do czynienia z samopodobieństwem statystycznym: rozkłady cech strukturalnych są podobne na różnych skalach, lecz nie zachodzi dokładna geometryczna odpowiedniość [5, s. xxi].

Pojęcie *wieloskalowości* jest ściśle związane z samopodobieństwem i oznacza brak charakterystycznej skali długości: w przeciwieństwie do gładkich obiektów geometrycznych, dla których poniżej pewnej skali struktura staje się jednorodna, fraktale ujawniają nowe szczegóły przy każdym poziomie przybliżenia [7, rozdz. 2]. W zastosowaniach wizualizacyjnych ma to bezpośrednie znaczenie praktyczne: przybliżanie obrazu fraktala nigdy nie prowadzi do „wygładzenia" struktury — granica zbioru pozostaje równie złożona niezależnie od skali obserwacji.

10:8001998811

1.2. Wymiar fraktalny

Jednym z powodów, dla których fraktale są opisywane jako obiekty o „nietypowej geometrii", jest zagadnienie ich wymiaru. W geometrii klasycznej wymiar jest liczbą całkowitą: krzywa jest jednowymiarowa, powierzchnia — dwuwymiarowa, a bryła — trójwymiarowa. Jednak dla wielu fraktali takie rozróżnienie okazuje się zbyt ubogie, ponieważ stopień wypełniania przestrzeni przez zbiór leży pomiędzy intuicyjnymi kategoriami całkowitymi [5, rozdz. 2.3].

Formalnym narzędziem do opisu tej własności jest *wymiar pudełkowy* (ang. *box-counting dimension*), pozwalający przypisać zbiorowi niekoniecznie całkowitą wartość wymiaru [5, rozdz. 3.1]:

$$D_B = \lim_{\varepsilon \to 0} \frac{\log N(\varepsilon)}{\log(1/\varepsilon)} \tag{1.2}$$

gdzie $N(\varepsilon)$ oznacza minimalną liczbę kwadratów (w przypadku dwuwymiarowym) o boku ε potrzebnych do pokrycia rozważanego zbioru. Dla trójkąta Sierpińskiego wartość ta wynosi $D_B \approx 1{,}585$, a dla płatka Kocha $D_B \approx 1{,}262$ — obydwie leżą między wymiarem krzywej a wymiarem powierzchni [7, rozdz. 4.3].

W praktyce obliczeniowej wymiar pudełkowy odpowiada sposobowi dyskretyzacji obrazu przez siatkę pikseli, co pozwala interpretować fraktale jako specyficzny typ skalowania złożoności reprezentacji cyfrowej.

1.3. Rys historyczny

Choć pojęcie fraktala zostało sformalizowane przez Benoita Mandelbrota dopiero w latach siedemdziesiątych XX wieku [2, rozdz. 1], obiekty o własnościach fraktalnych były badane przez matematyków znacznie wcześniej. W 1883 roku Georg Cantor opisał zbiór będący prototypem fraktala — nieskończenie cienki, lecz nieprzeliczalny podzbiór odcinka, powstały przez iteracyjne usuwanie środkowych trzecich części. Na przełomie XIX i XX wieku Helge von Koch (1904) skonstruował krzywą o nieskończonym obwodzie zamkniętą w skończonym obszarze, a Wacław Sierpiński (1915) wprowadził trójkąt i dywan noszące jego imię — obiekty, których wymiar jest niecałkowity [7, rozdz. 2].

Równolegle rozwijała się dynamika zespolona: Gaston Julia i Pierre Fatou niezależnie od siebie badali w latach 1917–1918 własności iteracji funkcji holomorficznych i opisali zbiory noszące dziś imię Julii. Praca ta wyprzedzała komputerowe wizualizacje o ponad pół wieku — pierwsze rysunki zbiorów Julii sporządzano ręcznie lub za pomocą wczesnych ploterów, co czyniło eksplorację tych obiektów niezwykle żmudną [4, s. 10].

Przełomem było wprowadzenie terminu „fraktal" przez Mandelbrota w 1975 roku [2, rozdz. 1] oraz opublikowanie przez niego w 1980 roku wyników dotyczących zbioru nazwanego później jego imieniem. Upowszechnienie komputerów osobistych w latach osiemdziesiątych sprawiło, że generowanie i eksploracja fraktali stały się dostępne szerokiemu gronu odbiorców. Michael Barnsley rozwinął w tym okresie teorię iterowanych układów funkcji (IFS), która umożliwiła zwarte kodowanie złożonych struktur naturalnych — takich jak paprocie czy chmury — za pomocą niewielkiej liczby przekształceń afinicznych [3, rozdz. 1].

1.4. Podstawy matematyczne

Matematyczny aparat wykorzystywany w opisie i generowaniu fraktali iteracyjnych opiera się na trzech powiązanych elementach: rachunku liczb zespolonych, pojęciu iteracji jako dys-

11:2893204013

kretnego układu dynamicznego oraz kryterium „ucieczki", pozwalającym odróżniać punkty o orbitach ograniczonych od tych, które dążą do nieskończoności [4, s. 8–15].

1.4.1. Liczby zespolone

Liczby zespolone zapisuje się w postaci

$$z = a + bi \tag{1.3}$$

gdzie $a, b \in \mathbb{R}$, zaś i jest jednostką urojoną spełniającą $i^2 = -1$. Część rzeczywista i urojona to odpowiednio $\mathrm{Re}(z) = a$ oraz $\mathrm{Im}(z) = b$. Liczbę zespoloną utożsamia się z punktem (a, b) na płaszczyźnie, co nadaje operacjom algebraicznym naturalną interpretację geometryczną.

W analizie iteracji kluczowe znaczenie ma moduł liczby zespolonej, rozumiany jako odległość punktu z od zera:

$$|z| = \sqrt{a^2 + b^2} \tag{1.4}$$

Wprowadza się również sprzężenie zespolone $\overline{z} = a - bi$, dla którego zachodzi $|z|^2 = z\overline{z}$. Z tych własności wynika nierówność trójkąta

$$|z + w| \leqslant |z| + |w| \tag{1.5}$$

przydatna w wyprowadzeniu kryterium ucieczki dla iteracji wielomianowych. W szczególności, dla rodziny $f_c(z) = z^2 + c$ z $|c| \leqslant 2$, jeżeli $|z_n| > 2$, to $|z_{n+1}| > |z_n|^2 - |c| > |z_n|$, co gwarantuje monotoniczny wzrost modułu i dążenie orbity do nieskończoności [7, rozdz. 13.4].

Dla $z \neq 0$ wygodna jest również postać biegunowa:

$$z = r(\cos\theta + i\sin\theta) = re^{i\theta} \tag{1.6}$$

gdzie $r = |z|$ jest modułem, a θ — argumentem. Przy mnożeniu moduły się mnożą, a argumenty sumują, zatem dla potęg zachodzi $z^n = r^n e^{in\theta}$. W kontekście fraktali opartych na wielomianach (np. rodzina $z^2 + c$) oznacza to, że każda iteracja kwadratowa bardzo szybko zwiększa moduł po przekroczeniu pewnego progu, co uzasadnia stosowanie kryterium ucieczki.

1.4.2. Iteracja i orbity

Niech $f\colon \mathbb{C} \to \mathbb{C}$ będzie funkcją zespoloną. Dla ustalonego punktu startowego z_0 definiuje się ciąg

$$z_{n+1} = f(z_n), \quad n = 0, 1, 2, \ldots \tag{1.7}$$

Ciąg $\{z_n\}_{n\geqslant 0}$ nazywa się *orbitą* punktu z_0 pod działaniem f [4, s. 27]. Kluczowe jest pytanie o asymptotyczne zachowanie orbity: czy pozostaje ona ograniczona, czy też jej moduł dąży do nieskończoności. To rozróżnienie leży u podstaw definicji większości klasycznych zbiorów fraktalnych.

1.4.3. Zbiory Julii i zbiór Mandelbrota

Dla rodziny wielomianów kwadratowych $f_c(z) = z^2 + c$, parametryzowanej przez $c \in \mathbb{C}$, definiuje się *wypełniony zbiór Julii*:

$$K_c = \left\{ z_0 \in \mathbb{C} : \sup_{n\geqslant 0} |f_c^{\,n}(z_0)| < \infty \right\} \tag{1.8}$$

12:4161872530

gdzie f_c^n oznacza n-krotne złożenie funkcji f_c [4, rozdz. 2]. *Zbiór Julii* J_c jest brzegiem K_c:

$$J_c = \partial K_c \tag{1.9}$$

Każda wartość parametru c definiuje inny zbiór Julii. Dla c leżącego wewnątrz *zbioru Mandelbrota* odpowiadający K_c jest spójny; dla c na zewnątrz — K_c jest zbiorem o strukturze Cantora (zdezintegrowanym) [4, s. 28].

Zbiór Mandelbrota definiuje się jako zbiór tych parametrów c, dla których orbita punktu $z_0 = 0$ pozostaje ograniczona:

$$\mathcal{M} = \left\{ c \in \mathbb{C} : \sup_{n \geqslant 0} |f_c^n(0)| < \infty \right\} \tag{1.10}$$

Zbiór ten można zatem interpretować jako „atlasową mapę" zbiorów Julii: przynależność c do $\mathcal{M}$ jest równoważna spójności K_c [7, rozdz. 14.2].

W obliczeniach numerycznych nieskończona liczba iteracji jest zastąpiona przez kryterium ucieczki: można wykazać, że jeśli $|z_n| > 2$ dla pewnego n, to orbita na pewno dąży do nieskończoności [7, rozdz. 14.1]. Warunek ten wyraża się równoważnie jako $|z_n|^2 > 4$, co eliminuje potrzebę obliczania pierwiastka kwadratowego w każdej iteracji.

1.5. Klasyfikacja fraktali

Z perspektywy implementacji programistycznej wygodny jest podział fraktali na trzy klasy, różniące się sposobem generowania obrazu.

Fraktale escape-time. W tej klasie obraz powstaje przez badanie każdego piksela z osobna: wyznaczany jest odpowiadający mu punkt c na płaszczyźnie zespolonej i uruchamiana jest iteracja aż do ucieczki lub osiągnięcia maksymalnej liczby kroków. Wynikiem jest wartość liczbowa (liczba iteracji do ucieczki) przypisywana pikselowi, a następnie mapowana na kolor. Do tej klasy należą zbiór Mandelbrota, zbiory Julii oraz Burning Ship [7, rozdz. 13.4].

Fraktale oparte na iterowanych układach funkcji (IFS). Iterowany układ funkcji to skończona rodzina odwzorowań zwężających $\{T_1, \ldots, T_k\}$, z których każde jest przekształceniem afinicznym płaszczyzny [3, rozdz. 3.7]. Zgodnie z twierdzeniem Banacha o punkcie stałym, istnieje unikatowy zwarty zbiór A (nazywany *atraktorem* układu) spełniający

$$A = \bigcup_{i=1}^{k} T_i(A) \tag{1.11}$$

W metodzie *chaos game* atraktor przybliżany jest przez generowanie długiego ciągu punktów: w każdym kroku losowane jest jedno z przekształceń (z zadanymi prawdopodobieństwami) i stosowane do bieżącego punktu. Po odpowiedniej liczbie kroków rozkład punktów zbiega do atraktora [3, s. 82]. Do tej klasy należą trójkąt Sierpińskiego i paproć Barnsleya.

Fraktale geometryczne (rekurencyjne). Trzecia klasa obejmuje obiekty konstruowane przez iteracyjne stosowanie reguły geometrycznej: płatek Kocha, krzywa Hilberta, dywan Sierpińskiego. Obiekty te nie są zaimplementowane w opisywanej aplikacji, jednak są ważnym punktem odniesienia w klasyfikacji fraktali i historycznie poprzedzają pozostałe dwie klasy [5, rozdz. 9].

Rozdział 2

Algorytmy generowania fraktali

Generowanie fraktali w aplikacji opiera się na dwóch fundamentalnie różnych podejściach obliczeniowych, których wybór wynika z natury matematycznej konkretnego fraktala. Pierwsza grupa — fraktale *escape-time* — obejmuje zbiór Mandelbrota, zbiór Julii oraz Burning Ship. W tej grupie obraz powstaje przez badanie, dla każdego punktu siatki pikseli, czy odpowiadająca mu orbita w przestrzeni zespolonej pozostaje ograniczona, czy też ucieka do nieskończoności. Druga grupa — fraktale punktowe oparte na iterowanych układach funkcji (IFS, ang. *Iterated Function System*) — obejmuje trójkąt Sierpińskiego i paproć Barnsleya. Tutaj algorytm generuje kolejne punkty metodą losowych przekształceń afinicznych, a obraz powstaje przez rasteryzację uzyskanej chmury punktów na siatkę pikseli.

Rozróżnienie między tymi dwiema klasami jest istotne nie tylko z matematycznego, ale i z implementacyjnego punktu widzenia, ponieważ prowadzi do odmiennych potoków renderowania, innych charakterystyk wydajnościowych i innych sposobów parametryzacji.

2.1. Ogólny opis algorytmu fraktalnego

Wspólnym elementem wszystkich zaimplementowanych fraktali jest pojęcie *siatki iteracyjnej* — dwuwymiarowej tablicy wartości zmiennoprzecinkowych o wymiarach $H \times W$, gdzie H i W to odpowiednio wysokość i szerokość obrazu w pikselach. Każdy element siatki przechowuje wynik obliczeń dla odpowiadającego mu punktu ekranu. Siatka ta jest następnie przekazywana do modułu kolorowania, który odwzorowuje wartości numeryczne na barwy pikseli. Takie podejście pozwala na pełne rozdzielenie warstwy obliczeniowej (generowanie fraktala) od warstwy prezentacyjnej (kolorowanie), co jest jednym z założeń projektowych aplikacji.

W przypadku fraktali escape-time wartością przechowaną w siatce jest liczba iteracji, po których orbita danego punktu przekroczyła granicę ucieczki — ewentualnie wygładzona za pomocą formuły opisanej w podrozdziale 2.3.4. Dla fraktali punktowych wartością jest indeks punktu przypisanego do danego piksela.

Po podzieleniu indeksów przez liczbę wszystkich wygenerowanych punktów uzyskuje się znormalizowane wartości w przedziale $[0, 1]$, które są następnie odwzorowywane na paletę barwną. Wizualnym efektem takiego kolorowania jest zakodowanie kolejności iteracji algorytmu chaos game w gradiencie barwnym — punkty wygenerowane wcześniej i później otrzymują różne barwy, dając obraz o ciągłej, organicznej strukturze.

Odwzorowanie piksela na płaszczyznę zespoloną

Fundamentem obliczeń escape-time jest przypisanie każdemu pikselowi (x, y) odpowiadającego mu punktu na płaszczyźnie zespolonej. Odwzorowanie realizowane jest przez obiekt kamery, który przechowuje współrzędne zespolone piksela w prawym dolnym rogu kadru (o_x, o_y)

14:1103198724

oraz aktualną skalę obserwacji s. Dla piksela o współrzędnych ekranowych (x, y) odpowiadający punkt zespolony $c = c_{\mathrm{re}} + ic_{\mathrm{im}}$ wyznaczany jest wzorami:

$$c_{\mathrm{re}} = (x - W) \cdot s + o_x \tag{2.1}$$

$$c_{\mathrm{im}} = (y - H) \cdot s + o_y \tag{2.2}$$

Zmiana skali s odpowiada przybliżaniu i oddalaniu widoku; zmiana przesunięcia (o_x, o_y) — poruszaniu się po płaszczyźnie.

2.2. Fraktale escape-time a fraktale punktowe

Dwa zaimplementowane podejścia do generowania fraktali różnią się fundamentalnie naturą matematyczną, jak i sposobem realizacji obliczeniowej.

Fraktale escape-time. Algorytmy z tej grupy operują bezpośrednio na siatce pikseli. Każdy piksel jest przetwarzany niezależnie: wyznaczany jest odpowiadający mu punkt na płaszczyźnie zespolonej, po czym uruchamiana jest iteracja funkcji f. Obliczenia kończą się, gdy moduł iterowanej wartości przekroczy zadaną granicę ucieczki lub gdy zostanie osiągnięta maksymalna liczba iteracji $N_{\max}$. Wynikiem jest wartość przypisana danemu pikselowi siatki.

Fraktale punktowe (IFS / chaos game). Algorytmy z tej grupy nie operują na siatce pikseli — zamiast tego generują dużą liczbę punktów na płaszczyźnie. Każdy kolejny punkt powstaje przez losowe zastosowanie jednej z predefiniowanych transformacji afinicznych do punktu poprzedniego. Po wygenerowaniu całej chmury punktów wykonywana jest rasteryzacja: każdy punkt jest rzutowany na siatkę pikseli za pomocą transformacji kamery. Podejście to, znane jako *chaos game*, jest specyficzną realizacją iterowanego układu funkcji.

Tabela 2.1: Porównanie właściwości obu podejść do generowania fraktali.

Cecha	Escape-time	IFS / chaos game
Reprezentacja wyjścia	siatka iteracji	chmura punktów
Krok rasteryzacji	zbędny	wymagany
Złożoność	$O(W \cdot H \cdot N_{\max})$	$O(N_{\mathrm{pkt}})$
Parametr jakości	`max_iter`	`num_points`
Deterministyczność	pełna	zależna od ziarna

2.3. Pseudokod algorytmów i omówienie kroków obliczeń

2.3.1. Zbiór Mandelbrota

Zbiór Mandelbrota definiowany jest jako zbiór tych punktów $c \in \mathbb{C}$, dla których orbita punktu $z_0 = 0$ pod działaniem funkcji $f_c(z) = z^2 + c$ pozostaje ograniczona. W praktyce obliczeniowej stosuje się kryterium ucieczki: jeśli $|z_n| > R$ dla pewnego n, orbita na pewno dąży do nieskończoności. Dla rodziny $z^2 + c$ wystarczającą granicą jest $R = 2$, co odpowiada warunkowi $|z|^2 > 4$.

15:7117855409

Rozkładając $z = z_x + iz_y$ na część rzeczywistą i urojoną, unika się arytmetyki zespolonej i operuje wyłącznie na liczbach rzeczywistych:

$$z'_x = z_x^2 - z_y^2 + c_{\mathrm{re}} \tag{2.3}$$

$$z'_y = 2z_x z_y + c_{\mathrm{im}} \tag{2.4}$$

a warunek ucieczki przyjmuje postać $z_x^2 + z_y^2 > 4$.

Algorithm 1 Generowanie siatki zbioru Mandelbrota

Require: W, H — wymiary obrazu; o_x, o_y — przesunięcie kamery; s — skala; $N_{\max}$ — maks. liczba iteracji
Ensure: L — siatka wartości $H \times W$

for $y \leftarrow 0$ **to** $H - 1$ **do**
  for $x \leftarrow 0$ **to** $W - 1$ **do**
    $c_{\mathrm{re}} \leftarrow (x - W) \cdot s + o_x$
    $c_{\mathrm{im}} \leftarrow (y - H) \cdot s + o_y$
    $z_x \leftarrow 0; \quad z_y \leftarrow 0; \quad n \leftarrow 0$
    while $n < N_{\max}$ **and** $z_x^2 + z_y^2 \leqslant 4$ **do**
      $z_x,\ z_y \leftarrow z_x^2 - z_y^2 + c_{\mathrm{re}}, \quad 2z_x z_y + c_{\mathrm{im}}$
      $n \leftarrow n + 1$
    end while
    if $n < N_{\max}$ **then**
      $r \leftarrow \sqrt{z_x^2 + z_y^2}$
      $L[y, x] \leftarrow n - \log(\log r) \,/\, \log 2$ ▷ smooth coloring — patrz podrozdz. 2.3.4
    else
      $L[y, x] \leftarrow N_{\max}$
    end if
  end for
end for

2.3.2. Zbiór Julii

Zbiory Julii korzystają z tej samej reguły iteracji $z_{n+1} = z_n^2 + c$, lecz różni je rola zmiennych w stosunku do zbioru Mandelbrota. W zbiorze Mandelbrota punkt startowy jest ustalony ($z_0 = 0$), a parametr c odpowiada badanemu pikselowi. W zbiorze Julii jest odwrotnie: parametr c jest ustalonym, globalnym parametrem algorytmu, natomiast punkt startowy z_0 odpowiada pikselowi.

Każda wartość $c \in \mathbb{C}$ definiuje inny zbiór Julii J_c. Parametr c jest interaktywnie ustawiany przez użytkownika za pomocą suwaków części rzeczywistej i urojonej. Domyślna wartość $c = -0{,}8 + 0{,}156i$ generuje jedną z bardziej znanych postaci zbioru Julii.

2.3.3. Burning Ship

Burning Ship jest wariantem algorytmu escape-time, w którym przed każdym krokiem kwadratowym do części rzeczywistej i urojonej z stosowana jest wartość bezwzględna. Reguła iteracji przyjmuje postać:

$$z_{n+1} = \Big(|\mathrm{Re}(z_n)| + i\,|\mathrm{Im}(z_n)|\Big)^2 + c \tag{2.5}$$

16:9766340578

Algorithm 2 Generowanie siatki zbioru Julii

Require: $W, H, o_x, o_y, s, N_{\max}$; $c_{\mathrm{re}}, c_{\mathrm{im}}$ — stały parametr c
Ensure: L — siatka wartości $H \times W$

for $y \leftarrow 0$ **to** $H-1$ **do**
  for $x \leftarrow 0$ **to** $W-1$ **do**
    $z_x \leftarrow (x - W) \cdot s + o_x$ ▷ piksel jako z_0, nie jako c
    $z_y \leftarrow (y - H) \cdot s + o_y$
    $n \leftarrow 0$
    while $n < N_{\max}$ **and** $z_x^2 + z_y^2 \leqslant 4$ **do**
      $z_x,\, z_y \leftarrow z_x^2 - z_y^2 + c_{\mathrm{re}}, \quad 2 z_x z_y + c_{\mathrm{im}}$
      $n \leftarrow n + 1$
    end while
    $L[y, x] \leftarrow$ *[smooth coloring jak w Mandelbrot]*
  end for
end for

co w rozwinięciu na składowe daje:

$$z'_x = z_x^2 - z_y^2 + c_{\mathrm{re}} \tag{2.6}$$

$$z'_y = 2|z_x||z_y| + c_{\mathrm{im}} \tag{2.7}$$

Złożenie wartości bezwzględnych z operacją kwadratową sprawia, że orbity są „składane" ku osi rzeczywistej, co nadaje fraktalowi jego charakterystyczny kształt przypominający płonący okręt — widoczny szczególnie przy domyślnym kadrze.

Algorithm 3 Generowanie siatki Burning Ship

Require: $W, H, o_x, o_y, s, N_{\max}$
Ensure: L — siatka wartości $H \times W$

for $y \leftarrow 0$ **to** $H-1$ **do**
  for $x \leftarrow 0$ **to** $W-1$ **do**
    $c_{\mathrm{re}} \leftarrow (x - W) \cdot s + o_x; \quad c_{\mathrm{im}} \leftarrow (y - H) \cdot s + o_y$
    $z_x \leftarrow c_{\mathrm{re}}; \quad z_y \leftarrow c_{\mathrm{im}}; \quad n \leftarrow 0$
    while $n < N_{\max}$ **and** $z_x^2 + z_y^2 \leqslant 4$ **do**
      $z'_x \leftarrow z_x^2 - z_y^2 + c_{\mathrm{re}}$
      $z'_y \leftarrow 2|z_x||z_y| + c_{\mathrm{im}}$
      $z_x, z_y \leftarrow z'_x, z'_y$
      $n \leftarrow n + 1$
    end while
    $L[y, x] \leftarrow$ *[smooth coloring]*
  end for
end for

2.3.4. Wygładzanie kolorowania iteracyjnego

Bezpośrednie użycie liczby iteracji n jako wartości piksela prowadzi do widocznych pasm — obszarów, w których wszystkie piksele mają tę samą całkowitoliczbową wartość. Zjawisko to jest szczególnie uciążliwe w przejściach między strefami o bliskich, lecz różnych wartościach n.

17:4956864849

Aby uzyskać ciągłe przejścia barwne, stosuje się formułę wygładzonego kolorowania iteracyjnego (ang. *smooth coloring*, *normalized iteration count*):

$$\mu = n - \frac{\log(\log r)}{\log 2} \tag{2.8}$$

gdzie $r = \sqrt{z_x^2 + z_y^2}$ jest modułem z w chwili ucieczki. Korekta $\log(\log r)/\log 2$ jest mniejsza dla punktów uciekających przy dużym r (krótko po przekroczeniu granicy) i większa dla punktów uciekających blisko progu. Wynikiem jest wartość rzeczywista, którą można gładko interpolować w przestrzeni kolorów, eliminując widoczne granice między pasami iteracyjnymi.

Powyższy wzór jest uproszczonym wariantem klasycznego `normalized iteration count` postaci $n + 1 - log_2(log_2|z|)$ [8]. W zaimplementowanej wersji wewnętrzny logarytm dwójkowy zastąpiono logarytmem naturalnym, co nie wpływa na ciągłość uzyskanego kolorowania — zachowana jest własność płynnego przejścia między iteracjami, a różnica sprowadza się do stałego przesunięcia w przestrzeni wartości μ dla orbit uciekających z podobnym modułem.

Piksele leżące wewnątrz zbioru — tj. takie, których orbita nie uciekła przed osiągnięciem $N_{\max}$ — otrzymują wartość $\mu = N_{\max}$ i są kolorowane jednolicie.

2.3.5. Trójkąt Sierpińskiego — chaos game

Trójkąt Sierpińskiego generowany jest metodą *chaos game*, będącą szczególnym przypadkiem iterowanego układu funkcji (IFS). Zamiast analizować przynależność punktu do zbioru, algorytm konstruuje zbiór przez iteracyjne generowanie punktów.

Wyróżnione zostają trzy wierzchołki równobocznego trójkąta:

$$v_0 = (0,\, 0), \quad v_1 = (1,\, 0), \quad v_2 = \left(\tfrac{1}{2},\, \tfrac{\sqrt{3}}{2}\right) \tag{2.9}$$

Algorytm startuje ze środka ciężkości trójkąta i w każdym kroku losuje jeden z wierzchołków, po czym przesuwa bieżący punkt o połowę drogi w jego kierunku:

$$p_{n+1} = \frac{p_n + v_k}{2} \tag{2.10}$$

gdzie $k \in \{0, 1, 2\}$ jest losowym indeksem. Po dostatecznie wielu iteracjach zbiór wygenerowanych punktów zbiega do atraktora IFS — trójkąta Sierpińskiego [3, s. 92].

Algorithm 4 Chaos game — trójkąt Sierpińskiego

Require: N — liczba punktów; *seed* — ziarno losowości
Ensure: P — tablica N punktów (x, y)
1: Ustaw ziarno generatora na *seed*
2: $(x, y) \leftarrow$ środek ciężkości v_0, v_1, v_2
3: **for** $i \leftarrow 0$ **to** $N - 1$ **do**
4: Losuj $k \in \{0, 1, 2\}$ jednostajnie
5: $x \leftarrow (x + v_k[0]) \cdot 0{,}5$
6: $y \leftarrow (y + v_k[1]) \cdot 0{,}5$
7: $P[i] \leftarrow (x,\, y)$
8: **end for**

Wynik jest w pełni deterministyczny dla ustalonego ziarna, co umożliwia zapamiętywanie (memoizację) wyniku i unikanie powtórnych obliczeń przy braku zmiany parametrów — mechanizm ten jest wykorzystany w implementacji.

18:8861799508

2.3.6. Paproć Barnsleya — iterowany układ funkcji (IFS)

Paproć Barnsleya jest klasycznym [3, rozdz. 1–2] przykładem atraktora iterowanego układu funkcji. Zdefiniowane są cztery przekształcenia afiniczne $T_1, \ldots, T_4$, z których każde ma postać:

$$T_k(x, y) = \begin{pmatrix} a_k & b_k \\ c_k & d_k \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} e_k \\ f_k \end{pmatrix} \tag{2.11}$$

Każde przekształcenie stosowane jest z prawdopodobieństwem p_k. Współczynniki użyte w aplikacji zestawiono w tabeli 2.2.

Tabela 2.2: Kanoniczne [3, s. 87] współczynniki przekształceń afinicznych paproci Barnsleya.

k	a_k	b_k	c_k	d_k	e_k	f_k	p_k	Opis
1	0,00	0,00	0,00	0,16	0,00	0,00	0,01	podstawa łodygi
2	0,85	0,04	−0,04	0,85	0,00	1,60	0,85	wzrost wzdłuż łodygi
3	0,20	−0,26	0,23	0,22	0,00	1,60	0,07	lewy podlistek
4	−0,15	0,28	0,26	0,24	0,00	0,44	0,07	prawy podlistek

Algorithm 5 IFS — paproć Barnsleya

Require: N — liczba punktów; *seed*; reguły $T_1, \ldots, T_4$ z prawdopodobieństwami $p_1, \ldots, p_4$
Ensure: P — tablica N punktów
1: Ustaw ziarno na *seed*; $(x, y) \leftarrow (0, 0)$
2: Oblicz skumulowany rozkład prawdopodobieństwa $F_k = \sum_{j=1}^{k} p_j$
3: **for** $i \leftarrow 0$ **to** $N - 1$ **do**
4: Losuj $r \in [0, 1)$ jednostajnie
5: Wybierz k takie, że $F_{k-1} \leqslant r < F_k$
6: $(x, y) \leftarrow T_k(x, y)$
7: $P[i] \leftarrow (x, y)$
8: **end for**

Rasteryzacja chmury punktów

Po wygenerowaniu tablicy punktów P przez algorytm IFS lub chaos game konieczne jest jej przekształcenie na siatkę pikseli. Operacja rasteryzacji przebiega następująco:

1. Każdy punkt $(x, y) \in P$ jest przeliczany na współrzędne pikselowe zgodnie z aktualnym stanem kamery:

$$x_{\text{pix}} = W - 1 - \left\lfloor \frac{x + o_x}{s} \right\rfloor, \qquad y_{\text{pix}} = H - 1 - \left\lfloor \frac{y + o_y}{s} \right\rfloor \tag{2.12}$$

2. Punkty wykraczające poza obszar obrazu są odrzucane.
3. Każdemu pikselowi w zasięgu przypisywana jest wartość niezerowa (indeks punktu).

Gdy kilka punktów chmury rzutuje się na ten sam piksel, w siatce zachowywany jest indeks punktu zapisanego najpóźniej. Wynikowy gradient barwny koduje zatem moment ostatniego trafienia w piksel, nie skumulowaną liczbę odwiedzeń, co pozwala na wektoryzację rasteryzacji w pojedynczej operacji NumPy, bez użycia pętli.

19:6833499768

2.4. Parametryzacja i wpływ parametrów na wygląd fraktala

Maksymalna liczba iteracji (`max_iter`). Parametr dostępny dla wszystkich fraktali escape-time. Wyznacza górną granicę liczby kroków iteracji dla każdego piksela. Zwiększenie jego wartości pozwala na dokładniejsze rozróżnianie punktów leżących blisko granicy zbioru, ujawniając drobniejsze struktury — kosztem proporcjonalnie dłuższego czasu renderowania. Przy eksploracji małych obszarów (duże przybliżenie) zwiększenie `max_iter` jest niezbędne, ponieważ orbity punktów leżących blisko granicy potrzebują więcej kroków, aby uciec.

Parametr c zbioru Julii (`c_re`, `c_im`). Zmiana wartości c radykalnie zmienia kształt zbioru Julii. Dla c leżącego wewnątrz zbioru Mandelbrota odpowiadający zbiór Julii jest spójny; dla c na zewnątrz — zbiór Julii jest zdezintegrowany. Na granicy zbioru Mandelbrota J_c przyjmuje szczególnie złożone, samopodobne formy.

Ziarno losowości (`seed`). Dla fraktali IFS ziarno inicjalizuje generator liczb pseudolosowych. Pełni ono dwie funkcje: zapewnia powtarzalność wyników (te same parametry produkują identyczny obraz) oraz pozwala uniknąć powtórnego generowania chmury punktów dzięki memoizacji. W granicy dużej liczby punktów rozkład zbiega do tego samego atraktora niezależnie od ziarna, zatem na końcowy efekt wizualny ziarno wpływa tylko przy małych wartościach `num_points`.

Liczba punktów (`num_points`). Parametr jakości dla fraktali IFS. Większa liczba punktów zwiększa gęstość zamalowania atraktora. W aplikacji zakres wynosi od 1 000 do 10 000 000 punktów.

2.5. Złożoność obliczeniowa i czynniki wpływające na czas renderowania

Fraktale escape-time. Złożoność generowania pojedynczej klatki wynosi $O(W \cdot H \cdot N_{\max})$ w pesymistycznym przypadku, gdy wszystkie piksele wymagają pełnej liczby iteracji. W praktyce znaczna część pikseli ucieka przed osiągnięciem $N_{\max}$ (szczególnie poza zbiorem), co skraca czas obliczeń. Niemniej zwiększenie rozdzielczości lub maksymalnej liczby iteracji prowadzi do wzrostu czasu renderowania.

Fraktale IFS. Złożoność generowania wynosi $O(N_{\text{pkt}})$, natomiast rasteryzacja — $O(N_{\text{pkt}} + W \cdot H)$. Generowanie punktów dominuje w typowych warunkach; rasteryzacja może stać się wąskim gardłem jedynie dla bardzo wysokich rozdzielczości przy małej liczbie punktów.

Kompilacja JIT. Wszystkie wewnętrzne pętle obliczeniowe zostały opatrzone dekoratorem `@jit(nopython=True)` z biblioteki Numba [11], co powoduje kompilację kodu Pythona do natywnego kodu maszynowego przy pierwszym wywołaniu. Efektywna wydajność jest wówczas porównywalna z implementacją w języku C. Pierwsze uruchomienie każdego algorytmu wiąże się z opóźnieniem kompilacji (rzędu kilku sekund), lecz kolejne wywołania korzystają z wersji skompilowanej.

20:1742946458

Memoizacja wyników IFS. Dla fraktali punktowych implementacja stosuje dodatkową memoizację (LRU cache, `@lru_cache(maxsize = 10)`): wynik generowania chmury punktów dla zadanej pary (`seed`, `num_points`) jest zapamiętywany i zwracany bezpośrednio przy kolejnym wywołaniu. Cache obejmuje 10 ostatnich kombinacji parametrów, dzięki czemu naprzemienne przełączanie między kilkoma zestawami parametrów również korzysta z zapamiętanych wyników. Zmiana stanu kamery (przesunięcie, przybliżenie) nie wymaga regeneracji punktów — powtórzony jest jedynie krok rasteryzacji.

2.6. Przykłady efektów wizualnych i powiązania między fraktalami

Mandelbrot a Julia. Zbiór Mandelbrota można interpretować jako „atlasową mapę" zbiorów Julii: każdy punkt c płaszczyzny parametrów odpowiada pewnemu zbiorowi Julii J_c. Punkty leżące wewnątrz zbioru Mandelbrota odpowiadają spójnym zbiorom Julii; punkty zewnętrzne — zbiorom zdezintegrowanym (kanoniczna struktura Cantora). Granica zbioru Mandelbrota odpowiada zbiorom Julii o szczególnie rozbudowanej strukturze fraktalnej.

Burning Ship a Mandelbrot. Burning Ship korzysta z tej samej rodziny wielomianów z^2+c, lecz z modyfikacją wartości bezwzględnych przed kwadratowaniem. Wprowadza to symetrię odbiciową względem osi rzeczywistej i nadaje fraktalowi asymetryczny charakter. W skali globalnej struktura zewnętrzna przypomina zbiór Mandelbrota, jednak szczegóły granicy są jakościowo odmienne i nie dają się do niego redukować.

Trójkąt Sierpińskiego a IFS. Trójkąt Sierpińskiego może być skonstruowany na dwa sposoby: geometrycznie (przez iteracyjne usuwanie środkowych trójkątów), lub przez algorytm chaos game. Obie metody zbiegają do tego samego obiektu fraktalnego, jednak chaos game jest implementacyjnie prostszy i wydajniejszy obliczeniowo dla dużych rozdzielczości.

Rozdział 3

Założenia projektowe i wymagania aplikacji

Niniejszy rozdział opisuje wymagania stawiane aplikacji oraz założenia przyjęte na etapie projektowania. Punktem wyjścia jest zdefiniowanie scenariuszy użycia, które wyznaczają zakres funkcjonalności. Na tej podstawie sformułowane zostają wymagania funkcjonalne i niefunkcjonalne wraz z ich priorytetami. Rozdział kończy się opisem docelowego środowiska użytkownika.

3.1. Scenariusze użycia i założenia funkcjonalne

Aplikacja skierowana jest do użytkownika zainteresowanego eksploracją fraktali — zarówno w celach poznawczych, jak i estetycznych. Zidentyfikowano cztery główne scenariusze użycia.

Eksploracja wizualna fraktala. Użytkownik uruchamia aplikację, wybiera jeden z dostępnych algorytmów i swobodnie porusza się po obrazie za pomocą klawiatury: klawisze `W/A/S/D` przesuwają widok, a `Q/E` obsługują przybliżanie i oddalanie. Interfejs reaguje natychmiastowo na wciśnięcie klawisza — obraz jest przerysowywany w sposób ciągły przez cały czas trwania ruchu.

Modyfikacja parametrów algorytmu. Użytkownik dostosowuje parametry aktywnego fraktala za pomocą suwaków dostępnych w panelu bocznym: maksymalną liczbę iteracji (fraktale escape-time), stałą c (zbiór Julii) lub liczbę punktów i ziarno losowości (fraktale IFS). Zmiana dowolnego parametru powoduje natychmiastowe przerysowanie obrazu.

Dobór kolorowania i zapis zrzutu ekranu. Użytkownik wybiera paletę barwną spośród dostępnych presetów lub definiuje własny gradient. Po uzyskaniu satysfakcjonującego efektu wizualnego zapisuje zrzut ekranu do pliku.

Tworzenie i eksport animacji. Użytkownik rejestruje serię klatek kluczowych (keyframes) odpowiadających różnym pozycjom i skalom kamery, wybiera tryb interpolacji (liniowy lub wygładzony krzywa Catmull-Rom) oraz docelowy format wyjściowy (GIF, MP4, WebM). Aplikacja generuje animację i zapisuje ją do pliku.

3.2. Wymagania i priorytety

Na podstawie scenariuszy użycia sformułowano wymagania funkcjonalne i niefunkcjonalne. Każde wymaganie oznaczono priorytetem: **W** (wymagane), **O** (opcjonalne).

22:2279638495

Wymagania funkcjonalne

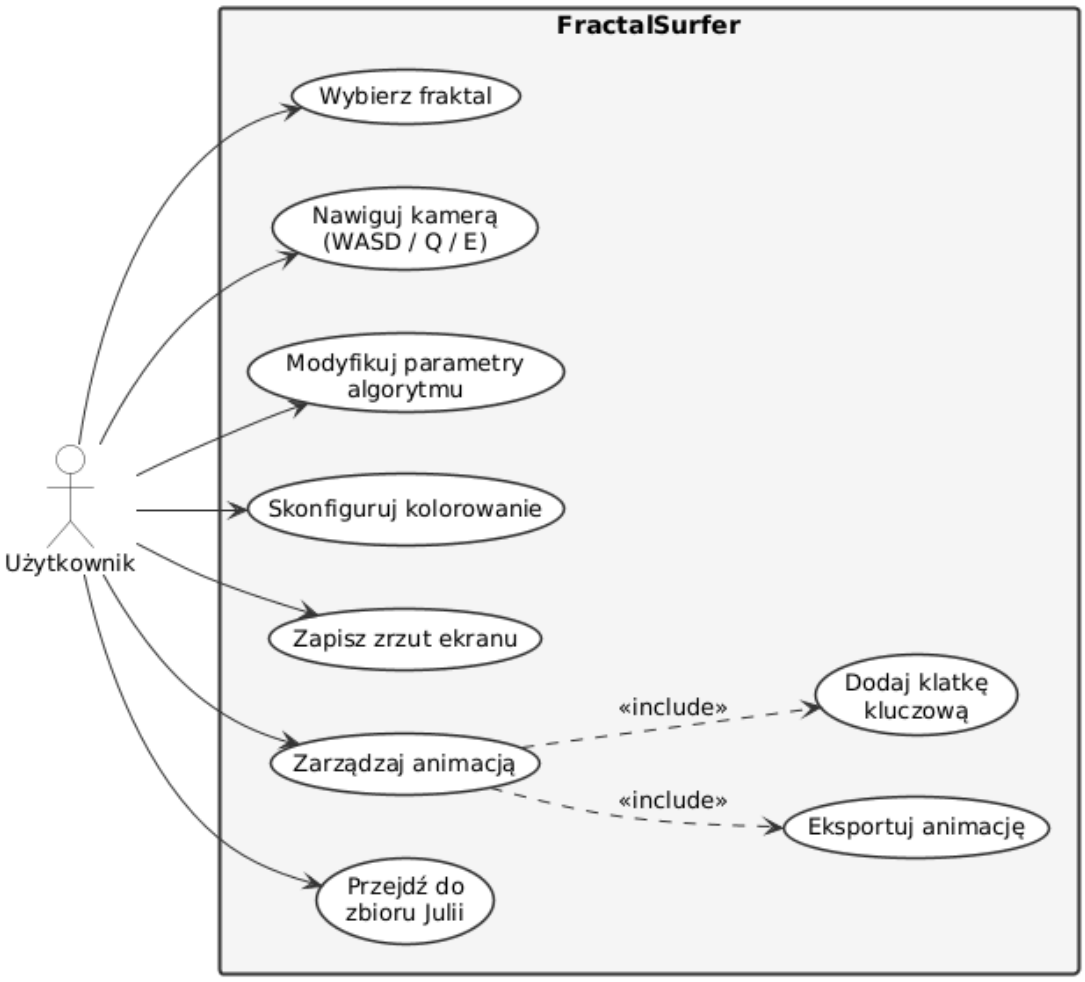
W Generowanie i wyświetlanie fraktali: zbioru Mandelbrota, zbioru Julii, Burning Ship, trójkąta Sierpińskiego, paproci Barnsleya.
W Nawigacja po obrazie za pomocą klawiatury (przesunięcie, przybliżanie i oddalanie).
W Interaktywna zmiana parametrów algorytmu z natychmiastowym efektem na renderowanym obrazie.
W Wybór schematu kolorowania spośród predefiniowanych presetów.
W Zapis bieżącego widoku do pliku graficznego (zrzut ekranu).
W Dynamiczne skalowanie interfejsu wraz ze zmianą rozmiaru okna aplikacji.
O Definiowanie własnej palety barwnej przez użytkownika.
O Tworzenie animacji opartej na klatkach kluczowych z interpolacją ruchu kamery.
O Eksport animacji do formatów GIF, MP4 i WebM.

Wymagania niefunkcjonalne

W **Responsywność interfejsu.** Sterowanie kamerą musi być płynne i nie może blokować wątku głównego, nawet podczas kosztownych obliczeń. Użytkownik powinien odczuwać natychmiastową reakcję na wciśnięcie klawisza.
W **Adaptacja jakości do możliwości sprzętowych.** Podczas aktywnego ruchu kamery dopuszczalne jest tymczasowe obniżenie jakości obrazu (mniejsza rozdzielczość, mniejsza liczba iteracji) w celu utrzymania płynności. Po zakończeniu ruchu renderowany jest obraz w pełnej jakości.
W **Minimalna rozdzielczość okna.** Okno aplikacji musi obsługiwać rozmiar co najmniej 1200×800 pikseli i umożliwiać swobodne powiększanie.
O **Przenośność.** Aplikacja powinna działać na systemach Windows, macOS i Linux bez modyfikacji kodu.
O **Deterministyczność eksportu.** Dla ustalonych parametrów i tych samych klatek kluczowych eksportowana animacja powinna być identyczna między uruchomieniami.

Diagram przypadków użycia (rys. 3.1) przedstawia wszystkie zidentyfikowane interakcje użytkownika z systemem. Przypadek użycia „Zarządzaj animacją" realizowany jest przez dwa podprzypadki w relacji `<<include>>`: „Dodaj klatkę kluczową" i „Eksportuj animację". Przypadek „Przejdź do zbioru Julii" dostępny jest wyłącznie w kontekście aktywnej zakładki Mandelbrota.

23:6074540611



Rysunek 3.1: Diagram przypadków użycia aplikacji FractalSurfer.

24:1084632955

3.3. Założenia procesu tworzenia projektu

Projekt realizowany był iteracyjnie: kolejne funkcjonalności były wdrażane i testowane niezależnie, a ich integracja następowała po weryfikacji poprawności działania każdego modułu z osobna. Punktem wyjścia było zaimplementowanie pojedynczego algorytmu (zbioru Mandelbrota) oraz minimalnego interfejsu umożliwiającego jego wyświetlenie. Dopiero po ustabilizowaniu podstawowego potoku renderowania — odwzorowanie pikseli na płaszczyznę zespoloną, obliczenia iteracyjne, kolorowanie, wyświetlanie tekstury — przystąpiono do rozszerzania projektu o kolejne fraktale, mechanizm nawigacji, system parametrów i moduł animacji.

Przyjęto kilka kluczowych założeń architektonicznych, które prowadziły ten proces:

— **Separacja logiki od interfejsu.** Algorytmy generowania fraktali nie zawierają żadnych odwołań do biblioteki GUI. Każdy algorytm implementuje jednolity interfejs (`FractalAlgorithm`) i operuje wyłącznie na danych geometrycznych (obiekt kamery) oraz parametrach domenowych. Dzięki temu algorytmy mogą być testowane i rozwijane niezależnie od warstwy wizualnej.
— **Reaktywność parametrów.** Każdy zestaw parametrów algorytmu dziedziczy po klasie `ReactiveParams`, która nakłada obowiązek dostarczenia metody budującej odpowiednie kontrolki GUI. Zmiana wartości parametru przez użytkownika automatycznie wyzwala przerysowanie obrazu bez konieczności ręcznego wiązania zdarzeń w kodzie warstwy wyższej.
— **Nieblokujące renderowanie.** Obliczenia fraktali oraz obsługa klawiatury zostały przeniesione do dedykowanych wątków demonicznych (`RedrawDaemon`, `KeyboardDaemon`), dzięki czemu pętla główna biblioteki DearPyGui pozostaje responsywna niezależnie od czasu trwania obliczeń.
— **Kompromis jakości i wydajności.** Zidentyfikowano, że podczas aktywnej nawigacji jakość obrazu jest mniej istotna niż szybkość reakcji. Mechanizm adaptacyjnego obniżania rozdzielczości i liczby iteracji (downscaling) jest aktywowany wyłącznie podczas ruchu i automatycznie wyłączany po jego zakończeniu.

3.4. Docelowe środowisko użytkownika

Aplikacja jest desktopową aplikacją okienkową przeznaczoną do uruchamiania lokalnie na komputerze osobistym. Wymagania środowiskowe są następujące:

— **System operacyjny:** Windows, macOS lub Linux (64-bitowy).
— **Interpreter:** Python w wersji co najmniej 3.12, wymaganej przez wykorzystane konstrukcje językowe (m.in. generyczne klasy bazowe w postaci `FractalAlgorithm[T]`).
— **Zależności:** DearPyGui (framework GUI), NumPy (operacje na tablicach), Numba (kompilacja JIT), Pillow i imageio (eksport obrazów i wideo). Zależności instalowane są w izolowanym środowisku wirtualnym.
— **Wymagania sprzętowe:** procesor wielordzeniowy jest zalecany, jednak nie jest wymagany — obliczenia prowadzone są w jednym wątku roboczym. Karta graficzna nie jest wykorzystywana; wszystkie obliczenia wykonywane są przez CPU.
— **Ekran:** minimalna rozdzielczość 1200×800 pikseli.

Aplikacja nie wymaga połączenia z internetem, instalacji systemowej ani uprawnień administratora. Uruchomienie sprowadza się do wywołania `python src/main.py` w aktywowanym środowisku wirtualnym.

Rozdział 4

Implementacja aplikacji

Rozdział opisuje szczegóły implementacyjne aplikacji: dobór technologii i uzasadnienie tych wyborów, architekturę systemu, sposób separacji warstw, mechanizmy komunikacji między komponentami oraz implementację wątków demonicznych odpowiedzialnych za responsywność interfejsu.

4.1. Wybór technologii

Python 3.12. Językiem implementacji jest Python w wersji co najmniej 3.12 [9]. Wybór ten podyktowany był dostępnością dojrzałych bibliotek naukowych i graficznych oraz zwięzłością kodu niezbędną do prototypowania i iteracyjnego rozwijania projektu. Wersja 3.12 była wymagana przez zastosowane konstrukcje językowe — w szczególności przez klasy generyczne bez importu z modułu `typing`, tj. składnię `FractalAlgorithm[T]`, która stała się dostępna w Pythonie 3.12.

DearPyGui. Jako framework graficzny wybrano DearPyGui [12] — bibliotekę opartą o model retained-mode z obsługą GPU (via Dear ImGui / DirectX 11 lub OpenGL). Biblioteka ta oferuje natywne wsparcie dla dynamicznych tekstur, będących podstawowym mechanizmem wyświetlania obrazu fraktala: aplikacja generuje tablicę pikseli po stronie CPU i przekazuje ją do tekstury DearPyGui, która jest następnie renderowana na ekranie przez GPU. DearPyGui dostarcza również gotowe kontrolki (suwaki, zakładki, przyciski, nagłówki składane), co wyeliminowało potrzebę ręcznej implementacji interfejsu użytkownika.

NumPy. Biblioteka NumPy [10] używana jest do reprezentacji siatek iteracyjnych i map kolorów jako wielowymiarowych tablic o ustalonym typie. Typ `Double` (`np.double`, tj. 64-bitowa liczba zmiennoprzecinkowa) jest konsekwentnie stosowany we wszystkich obliczeniach, eliminując niejawne konwersje typów, które mogłyby prowadzić do błędów numerycznych. Operacje na tablicach, takie jak rasteryzacja punktów IFS czy skalowanie w górę (upscaling), realizowane są przez wektoryzowane funkcje NumPy, co zapewnia wysoką wydajność bez pisania jawnych pętli w Pythonie.

Numba. Wszystkie wewnętrzne pętle obliczeniowe (iteracja escape-time, chaos game, IFS) są kompilowane do kodu natywnego przez bibliotekę Numba [11] za pomocą dedykowanego dekoratora `@jit(nopython=True)`. W trybie `nopython` Numba nie korzysta z interpretera Pythona — generowany kod maszynowy jest funkcjonalnie równoważny implementacji w C. Pierwsze wywołanie każdej skompilowanej funkcji wiąże się z jednorazowym narzutem kompilacji (kilka sekund); kolejne wywołania korzystają z wersji skompilowanej i są wykonywane z pełną prędkością.

26:3289247604

Pillow i imageio. Do zapisu zrzutów ekranu (format PNG) używana jest biblioteka Pillow [13]. Eksport animacji do formatów MP4 i WebM realizowany jest przez bibliotekę imageio [14] z backendami FFmpeg, natomiast zapis GIF-ów ponownie korzysta z Pillow.

4.2. Architektura systemu

Kod źródłowy podzielony jest na trzy główne pakiety, odpowiadające trzem warstwom odpowiedzialności:

— `common/` — typy i abstrakcje współdzielone przez pozostałe warstwy: definicje typów (`Double`, `Lattice`, `LUTtice`), klasa kamery (`Camera`) oraz bazowa abstrakcja reaktywnych parametrów (`ReactiveParams`).
— `fractal/` — warstwa logiki domenowej. Zawiera implementacje algorytmów fraktalnych oraz klasę bazową `FractalAlgorithm`. Moduły tej warstwy nie importują ani nie odwołują się do żadnego elementu warstwy interfejsu.
— `ui/` — warstwa interfejsu użytkownika. Odpowiada za budowanie i aktualizację okna aplikacji, obsługę zdarzeń, zarządzanie stanem oraz eksport wyników. Zawiera podpakiety `ui/daemons/` (wątki demoniczne) i `ui/widgets/` (widgety DearPyGui).

Centralnym punktem warstwy UI jest klasa `UIState`, która agreguje wszystkie elementy stanu aplikacji: aktualnie wybrany algorytm i jego parametry, obiekt kamery, parametry kamery, schemat kolorowania oraz parametry animacji. `UIState` pełni rolę pośrednika między warstwą logiki (algorytmy fraktali) a warstwą prezentacji (DearPyGui, `TexturePanel`).

Punkt wejścia aplikacji (`main.py`) tworzy instancję `UIState`, buduje drzewo widgetów DearPyGui, rejestruje handlery klawiatury i zmian rozmiaru okna, a następnie uruchamia wątki demoniczne i oddaje sterowanie pętli zdarzeń DearPyGui.

4.3. Separacja warstwy logicznej i interfejsu

Kluczowym założeniem projektowym jest ścisłe oddzielenie kodu odpowiedzialnego za obliczenia fraktali od kodu interfejsu użytkownika.

Klasa bazowa FractalAlgorithm

Każdy algorytm fraktalny implementuje abstrakcyjną klasę bazową:

— `default_camera(width, height)` → `Camera` — zwraca domyślną kamerę dopasowaną do wymiarów okna dla danego fraktala.
— `generate_frame(camera, params)` → `Lattice` — generuje siatkę iteracyjną dla zadanego stanu kamery i parametrów.

Klasa bazowa jest sparametryzowana typem parametrów: `FractalAlgorithm[T: ReactiveParams` co zapewnia spójność typów między implementacją algorytmu a jego parametrami. Żaden z modułów warstwy `fractal/` nie importuje elementów z warstwy `ui/`.

Klasa ReactiveParams

Parametry każdego algorytmu dziedziczą po abstrakcyjnej klasie `ReactiveParams`, która nakłada jeden obowiązek: dostarczenie metody `build_dpg_params(parent, setattr_)`, budującej kontrolki GUI przypisane do danego zestawu parametrów.

27:4952072190

Argument `setattr_` jest funkcją o sygnaturze zgodnej ze standardową funkcją `setattr`, która po każdej zmianie wartości atrybutu automatycznie wysyła sygnał przerysowania (`redraw()`). Dzięki temu każdy suwak w interfejsie jest trwale powiązany z przerysowaniem obrazu bez konieczności ręcznego definiowania tej zależności dla każdego parametru z osobna.

Enum AlgoChoice

Klasa `AlgoChoice` (enum) pełni rolę rejestru algorytmów: mapuje symboliczne nazwy (wyświetlane jako etykiety zakładek w GUI) na klasy implementacyjne i fabryki parametrów. Dodanie nowego algorytmu do aplikacji sprowadza się do dodania jednej wartości w tym enum oraz odpowiednich gałęzi w metodach `get_algo_impl` i `get_algo_params`.

4.4. Mechanizm komunikacji i przepływ zdarzeń

Przepływ zdarzeń w aplikacji przebiega przez trzy poziomy: wątku głównego DearPyGui, wątku `KeyboardDaemon` i wątku `RedrawDaemon`.

Zmiana parametru algorytmu. Użytkownik przesuwa suwak → DearPyGui wywołuje callback → `setattr_` ustawia nową wartość pola parametrów → `redraw()` ustawia flagę `redraw_event` → `RedrawDaemon` budzi się i generuje nową klatkę.

Nawigacja kamerą. Użytkownik wciska klawisz → DearPyGui wywołuje handler `on_key_down` → `add_key` dodaje klawisz do zbioru wciśniętych klawiszy w `KeyboardDaemon` i ustawia `wakeup_event` → `KeyboardDaemon` budzi się, w pętli aktualizuje pozycję kamery i po każdym kroku wywołuje `redraw()` → po zwolnieniu klawisza pętla kończy się i następuje finalne przerysowanie w pełnej jakości.

Zmiana algorytmu. Użytkownik klika zakładkę → DearPyGui wywołuje `on_tab_change` → `UIState.replace_algo` podmienia algorytm i resetuje kamerę → budowane są nowe kontrolki parametrów → `redraw()`.

Zmiana rozmiaru okna. Zdarzenie resize → `on_viewport_resize` → `UIState.handle_resize` aktualizuje wymiary `TexturePanel` i kamery → `redraw()`.

Centralną rolę w tym schemacie pełni funkcja `redraw()`, która jest wyłącznie sygnałem: nie wykonuje żadnych obliczeń, a jedynie ustawia zdarzenie `threading.Event` oczekiwane przez `RedrawDaemon`.

4.5. System wątków demonicznych

4.5.1. RedrawDaemon — odświeżanie obrazu

`RedrawDaemon` jest wątkiem demonicznym (`daemon=True`), który przez cały czas działania aplikacji czeka na sygnał `redraw_event`. Po jego otrzymaniu wykonuje następującą sekwencję:

1. Zapamiętuje aktualne wymiary obrazu.
2. Wywołuje `UIState.generate_frame()`, która generuje siatkę iteracyjną i nakłada na nią paletę barwną (wynik: `LUTtice`).

28:1003420385

3. Sprawdza, czy wymiary nie zmieniły się podczas obliczeń (zabezpieczenie przed wyścigiem danych przy jednoczesnym resize).
4. Jeśli wymiary są spójne, aktualizuje teksturę przez `TexturePanel.update()`.
5. Czyści flagę `redraw_event`.

Zastosowanie `threading.Event` zamiast aktywnego oczekiwania (busy wait) oznacza, że wątek nie zużywa zasobów CPU w czasie bezczynności.

4.5.2. Resize i zarządzanie teksturą

Klasa `TexturePanel` opakowuje dynamiczną teksturę DearPyGui. Przy każdej zmianie rozmiaru okna tworzona jest nowa tekstura o aktualnych wymiarach, a poprzednia jest usuwana. Wymiary panelu tekstury są obliczane przez odjęcie szerokości panelu bocznego i marginesów od wymiarów okna aplikacji.

Metoda `update` przyjmuje tablicę `LUTtice` (wymiary $H \times W \times 3$, typ `uint8`), dołącza pełny kanał alfa (wartość 255), a następnie normalizuje wartości do zakresu $[0, 1]$ i przekazuje spłaszczoną tablicę do DearPyGui.

4.5.3. KeyboardDaemon — obsługa klawiatury i ruchu

`KeyboardDaemon` zarządza zbiorem aktualnie wciśniętych klawiszy i odpowiada za płynną nawigację kamerą. Jego pętla robocza:

1. Czeka na `wakeup_event` (ustawiany przy każdym wciśnięciu klawisza nawigacyjnego).
2. Włącza tryb obniżonej jakości renderowania (`enable_movement_camera`).
3. W pętli, dopóki zbiór wciśniętych klawiszy jest niepusty:
   a) Aktualizuje pozycję i skalę kamery na podstawie aktualnie wciśniętych klawiszy.
   b) Wysyła sygnał `redraw()`.
   c) Oblicza czas trwania kroku i usypia na pozostały czas do następnego kroku (`polling_stagger`), zapewniając stałą częstotliwość aktualizacji.
4. Po zakończeniu pętli wyłącza tryb obniżonej jakości i wysyła finalne `redraw()`, aby uzyskać obraz w pełnej rozdzielczości.

Częstotliwość aktualizacji jest kontrolowana przez parametr nazywany w pracy jako `polling_stagg` — odstęp czasowy pomiędzy kolejnymi krokami pętli aktualizującej kamerę. Mechanizm ten decyduje o płynności nawigacji: zbyt mała wartość prowadzi do przeciążenia wątku renderującego, zbyt duża — do widocznych skoków obrazu.

4.6. Struktura kodu i klasy bazowe

Typy współdzielone (common/types.py)

Moduł `common/types.py` definiuje trzy typy używane w całym projekcie:

— `Double` (`np.double`) — 64-bitowa liczba zmiennoprzecinkowa. Stosowana jednolicie we wszystkich obliczeniach numerycznych w celu uniknięcia niejawnych konwersji.
— `Lattice` — dwuwymiarowa tablica `Double` ($H \times W$). Wynik działania algorytmu fraktalnego przed nałożeniem kolorowania.
— `LUTtice` — trójwymiarowa tablica `uint8` ($H \times W \times 3$). Wynik po nałożeniu palety barwnej; gotowy do wyświetlenia obraz RGB.

29:6283469853

Adaptacyjne obniżanie jakości (UIState.generate_frame)

Metoda `generate_frame` w `UIState` realizuje mechanizm adaptacyjnego obniżania jakości podczas ruchu. Gdy aktywny jest tryb nawigacji (flaga `movement_enabled_downscaling`) oraz użytkownik nie wyłączył tej funkcji (`user_enabled_downscaling`), metoda:

1. Tworzy tymczasową kamerę z wymiarami zmniejszonymi przez `downscale_factor` (np. dwukrotnie) i odpowiednio powiększoną skalą.
2. Tworzy tymczasowe parametry algorytmu z obniżoną liczbą iteracji (`iteration_limit`).
3. Generuje małą siatkę, a następnie skaluje ją z powrotem do pełnych wymiarów przez powielanie pikseli (`np.repeat`).

Skalowanie `np.repeat` (odpowiednik interpolacji najbliższego sąsiada) jest celowym uproszczeniem: podczas ruchu jakość obrazu jest tymczasowo obniżona, a szczegóły fraktala w pełnej rozdzielczości są renderowane dopiero po zakończeniu nawigacji.

4.7. Zarządzanie zależnościami

Projekt używa izolowanego środowiska wirtualnego Python (`.venv/`) zarządzanego przez standardowe narzędzie `venv`. Zależności zewnętrzne to: `dearpygui`, `numpy`, `numba`, `pillow` i `imageio`. Środowisko wirtualne zapewnia, że wersje bibliotek są stabilne i niezależne od instalacji systemowej Pythona, co jest szczególnie istotne w przypadku Numby, której kompatybilność z NumPy jest ściśle wersjonowana.

Rozdział 5

Projekt interfejsu użytkownika

To, w jaki sposób użytkownik i program komunikują się ze sobą, wpływa jednocześnie na komfort i funkcjonalność aplikacji. Rozdział omawia projekt interaktywnego interfejsu użytkownika, zarysowując wymagania i funkcje użytkowe potrzebne do pełnej interakcji z programem.

5.1. Założenia UX/UI

Interfejs użytkownika zaprojektowano według kilku nadrzędnych zasad. Po pierwsze, **minimalizm sterowania**: podstawowa eksploracja fraktala powinna być możliwa wyłącznie za pomocą klawiatury, bez konieczności używania myszy. Po drugie, **natychmiastowość sprzężenia zwrotnego**: każda zmiana parametru, pozycji kamery lub schematu kolorowania powinna natychmiast odzwierciedlać się w renderowanym obrazie. Po trzecie, **widoczność stanu**: użytkownik powinien zawsze widzieć aktualne współrzędne centrum widoku, aktywny algorytm i liczbę zarejestrowanych klatek kluczowych animacji.

Zdecydowano się na dwupanelowy układ okna: po lewej stronie dominuje panel obrazu (tekstura fraktala), a po prawej wąski panel boczny (400 pikseli) zawiera wszystkie kontrolki. Układ ten maksymalizuje powierzchnię podglądu przy jednoczesnym zachowaniu dostępu do wszystkich opcji w jednym widoku — bez konieczności otwierania dodatkowych okien czy zakładek.

5.2. Organizacja paneli

Panel obrazu (TexturePanel)

Panel obrazu zajmuje lewą część okna i automatycznie dostosowuje się do jego rozmiarów. Wyświetla on dynamiczną teksturę DearPyGui, aktualizowaną przy każdym zdarzeniu przerysowania. Wymiary tekstury są obliczane przez odjęcie szerokości panelu bocznego i stałych marginesów od bieżących wymiarów okna aplikacji.

Panel boczny

Panel boczny podzielony jest na cztery sekcje wyświetlane sekwencyjnie od góry:

1. **Pasek zakładek algorytmów** — wybór aktywnego fraktala. Każda zakładka odpowiada jednemu algorytmowi i zawiera jego specyficzne parametry (suwaki). Przełączenie zakładki powoduje zmianę algorytmu, reset kamery do domyślnej pozycji oraz wyczyszczenie klatek kluczowych animacji.
2. **Sekcja kamery** — ustawienia widoku i nawigacji, zapis zrzutu ekranu.
3. **Sekcja animacji** — zarządzanie klatkami kluczowymi i eksport.
4. **Sekcja kolorowania** — wybór i konfiguracja palety barwnej.

31:1062053670

Sekcje kamery, animacji i kolorowania są zwijalne (ang. *collapsing header*), co pozwala użytkownikowi ukryć nieistotne w danej chwili opcje i zachować przejrzystość interfejsu.

5.3. Sterowanie kamerą

Nawigacja po obrazie odbywa się wyłącznie za pomocą klawiatury. Zdefiniowane są następujące klawisze sterujące:

— `W`/`S` — przesunięcie widoku w górę / w dół,
— `A`/`D` — przesunięcie widoku w lewo / w prawo,
— `Q`/`E` — przybliżenie / oddalenie.

Klawisze modyfikatorów zmieniają szybkość nawigacji:
— `Shift` — mnożnik przyspieszenia (domyślnie $\times 1{,}5$),
— `Ctrl` — mnożnik spowolnienia (domyślnie $\times 0{,}5$).

Obie kombinacje można stosować jednocześnie z klawiszem ruchu lub zoomu. Gdy oba modyfikatory są wciśnięte równocześnie, wzajemnie się znoszą i stosowana jest prędkość bazowa.

Zoom realizowany jest przez skalowanie parametru kamery w dziedzinie liniowej: przybliżenie mnoży skalę przez stałą $z < 1$, oddalenie przez $1/z$. Operacja ta zachowuje środek widoku — nowe przesunięcie jest wyliczane tak, żeby punkt centralny kadru nie zmieniał się po zmianie skali.

Aktualnie wyświetlane współrzędne środka kadru (w układzie współrzędnych płaszczyzny zespolonej) są na bieżąco aktualizowane w panelu kamery jako etykieta tekstowa. Umożliwia to precyzyjną nawigację do znanych lokalizacji matematycznych (np. punkt $c = -0{,}75$ na osi rzeczywistej zbioru Mandelbrota, gdzie bifurkuje okres 2).

Wszystkie parametry sterowania są konfigurowalne przez panel kamery: prędkość ruchu, prędkość zoomu, wartości modyfikatorów oraz częstotliwość odpytywania klawiatury (*polling stagger*).

5.4. Funkcje użytkowe

Resetowanie kamery. Przycisk „reset camera” przywraca kamerę do domyślnej pozycji dla aktywnego algorytmu. Domyślne kadrowanie kamery dla każdego algorytmu zostało dobrane eksperymentalnie tak, aby przy uruchomieniu aplikacji prezentować charakterystyczny, rozpoznawalny widok danego fraktala.

Zrzut ekranu. Przycisk „save screenshot” zapisuje aktualną zawartość panelu obrazu do pliku PNG w katalogu `screenshots/`. Nazwa pliku zawiera znacznik czasu. Status operacji (nazwa pliku lub komunikat błędu) wyświetlany jest obok przycisku.

Przejście do odpowiadającego zbioru Julii. Zakładka Mandelbrota zawiera dodatkowy przycisk „go to corresponding Julia set”. Jego naciśnięcie odczytuje aktualne współrzędne środka kadru i ustawia je jako parametr c zbioru Julii, po czym automatycznie przełącza aktywną zakładkę na Julia. Pozwala to bezpośrednio eksplorować związek między pozycją w zbiorze Mandelbrota a kształtem odpowiadającego mu zbioru Julii.

Adaptacyjne obniżanie jakości. Checkbox „movement downscaling” włącza lub wyłącza mechanizm obniżania rozdzielczości i liczby iteracji podczas aktywnego ruchu kamery (opisany

szczegółowo w rozdziale 4). Użytkownik może regulować współczynnik zmniejszenia rozdzielczości (*downscale factor*, zakres 2–16) oraz maksymalną liczbę iteracji podczas ruchu (*iteration limit*).

33:7202999971

Rozdział 6

Kolorowanie fraktali

Siatka iteracyjna generowana przez algorytm fraktalny jest tablicą liczb rzeczywistych — sama w sobie nie niesie informacji o barwie. Dopiero nałożenie palety barwnej (ang. *look-up table*, LUT) przekształca wartości numeryczne w obraz wizualny. Sposób kolorowania ma zasadniczy wpływ na odbiór estetyczny i poznawczy fraktala: ta sama struktura matematyczna, pokolorowana inaczej, może sprawiać wrażenie ciepłej, organicznej formy lub zimnego, technicznego wzoru.

6.1. Algorytmy kolorowania

Normalizacja i LUT

Przed zastosowaniem palety wartości siatki są normalizowane do zakresu $[0, 1]$ przez podzielenie przez maksymalną liczbę iteracji $N_{\max}$ (lub liczbę punktów `num_points` dla fraktali IFS) i obcięcie do $[0, 1]$:

$$t = \mathrm{clip}\Big(\frac{\mu}{N_{\max}},\ 0,\ 1\Big) \tag{6.1}$$

Znormalizowana wartość t jest następnie mapowana na indeks tablicy LUT (o rozmiarze 2048 wpisów) i odczytywana jest odpowiadająca jej barwa RGB. Piksele należące do zbioru ($\mu = N_{\max}$) otrzymują kolor czarny niezależnie od wybranej palety.

Interpolacja gradientu

Paleta barwna jest zdefiniowana przez listę punktów kontrolnych (ang. *colour stops*): par $\{(t_k, \mathbf{c}_k)\}$, gdzie $t_k \in [0, 1]$ jest pozycją na osi wartości, a $\mathbf{c}_k = (r, g, b) \in [0, 1]^3$ — przypisaną barwą. Między kolejnymi punktami kontrolnymi kolor jest interpolowany liniowo w przestrzeni RGB:

$$\mathbf{c}(t) = (1 - w)\,\mathbf{c}_k + w\,\mathbf{c}_{k+1}, \qquad w = \frac{t - t_k}{t_{k+1} - t_k} \tag{6.2}$$

Korekcja gamma

Po interpolacji gradientu stosowana jest korekcja gamma:

$$\mathbf{c}_\gamma(t) = \mathbf{c}(t)^{1/\gamma} \tag{6.3}$$

gdzie $\gamma \in (0, 1]$ jest parametrem kontrolowanym przez użytkownika. Wartość $\gamma < 1$ rozjaśnia ciemne obszary (obszary graniczne fraktala), uwydatniając subtelne struktury. Wartość domyślna $\gamma = 1{,}0$ oznacza brak korekcji.

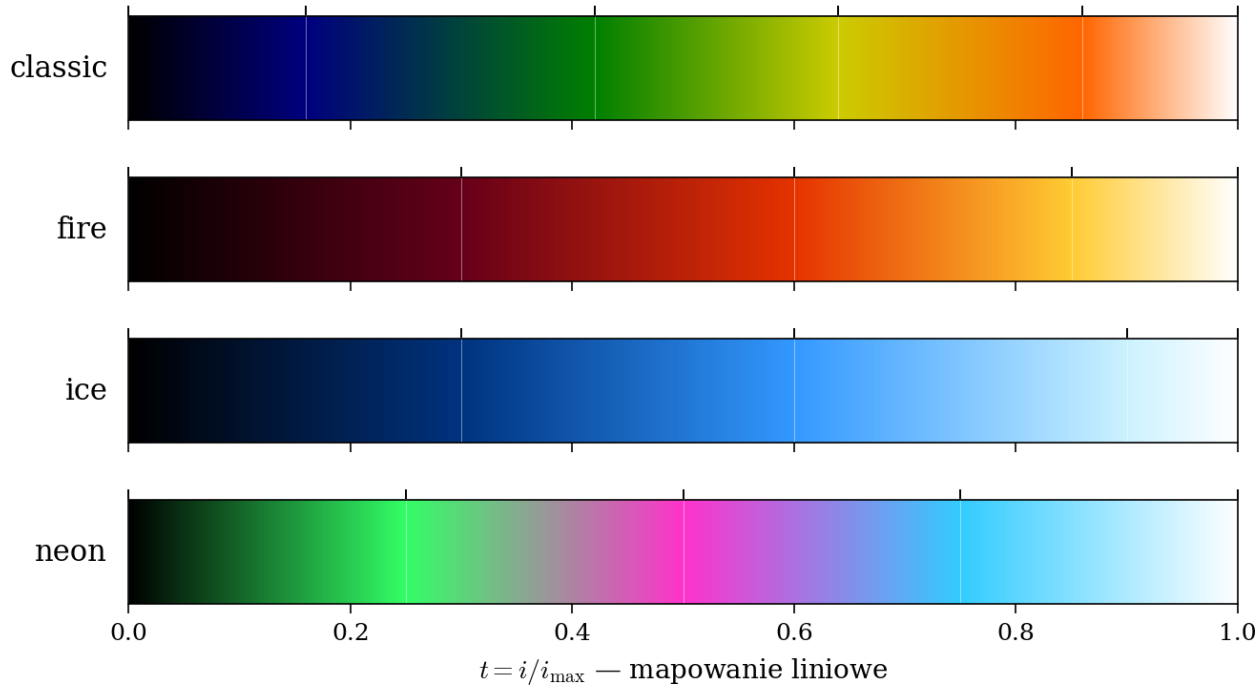
34:9968425719

6.2. Presety i kolorowanie niestandardowe

Tryb presetów

Aplikacja oferuje cztery predefiniowane palety barwne, przedstawione na rysunku 6.1:

— **classic** — czerń, granat, zieleń, żółć, pomarańcz, biel; nawiązuje do klasycznych wizualizacji zbioru Mandelbrota.
— **fire** — czerń, ciemna czerwień, jaskrawa pomarańcz, żółtawa biel; wyraźny motyw ognia.
— **ice** — czerń, ciemny niebieskogranat, jasny błękit, prawie biały; wrażenie lodu i zimna.
— **neon** — czerń, jasna zieleń, różowy magenta, błękit; paleta kontrastowa o silnym charakterze wizualnym.

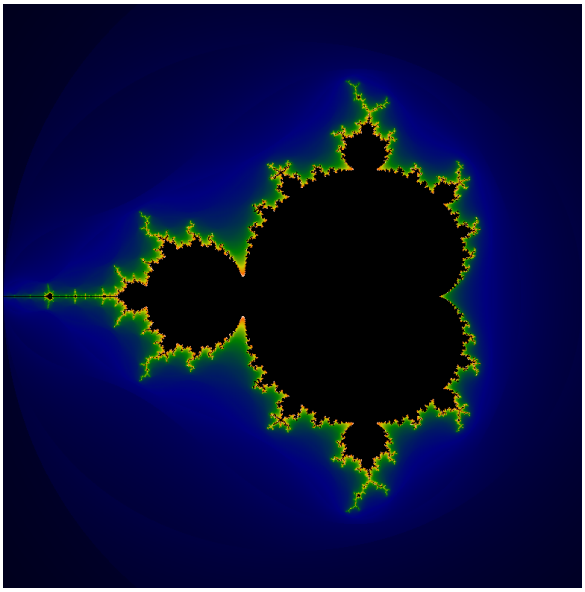


Rysunek 6.1: Przedstawienie predefiniowanych palet barwnych jako gradient liniowy. Punkty kontrolne zostały zaznaczone na górze każdego gradientu.

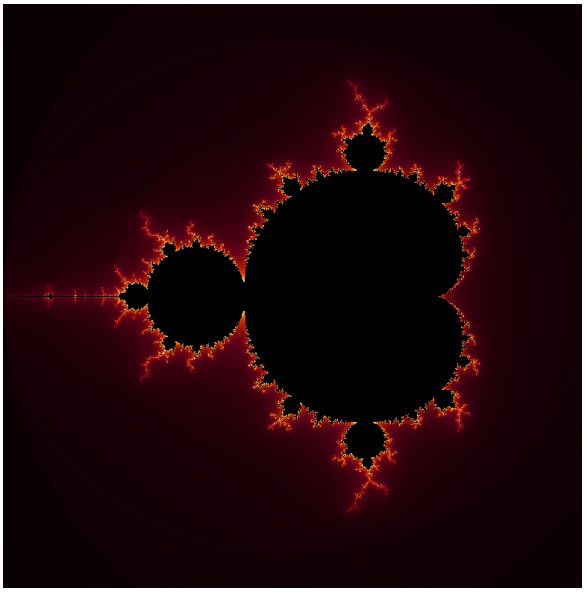
Należy pamiętać o naturze samych fraktali: paleta nałożona na fraktal daje efekt wyraźnie nierównomierny (rys. 6.2). Liczba pikseli uciekających po n krokach maleje w przybliżeniu wykładniczo wraz ze wzrostem n, więc po liniowej normalizacji $t = \mu/N_{\max}$ zdecydowana większość obrazu trafia w początkowy fragment palety, a końcowe odcienie zostają ściśnięte do wąskich pasm wzdłuż granicy zbioru. Zostało to uwzględnione podczas tworzenia presetów.

Tryb niestandardowy

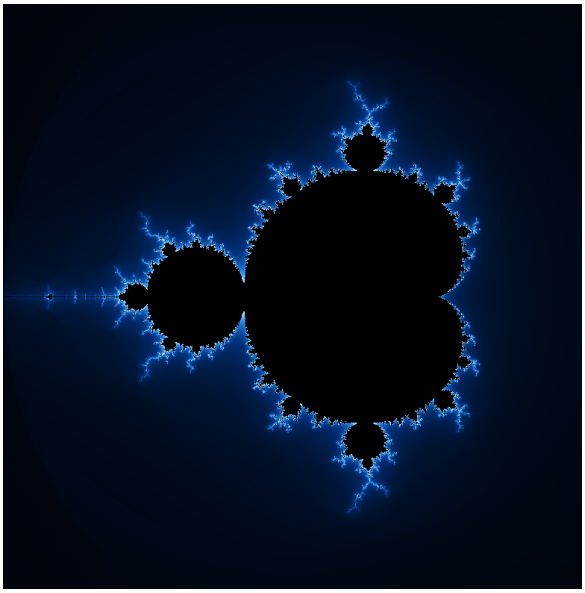
W trybie niestandardowym użytkownik definiuje własne punkty kontrolne bezpośrednio w interfejsie. Każdy punkt składa się z suwaka pozycji $t \in [0, 1]$ oraz selektora koloru RGB. Punkty można dynamicznie dodawać przyciskiem „add stop” i usuwać przyciskiem „delete” przy każdym z nich. Zmiana dowolnej wartości powoduje natychmiastowe przerysowanie obrazu. Porównanie trybów przedstawione jest na rysunku 6.3.



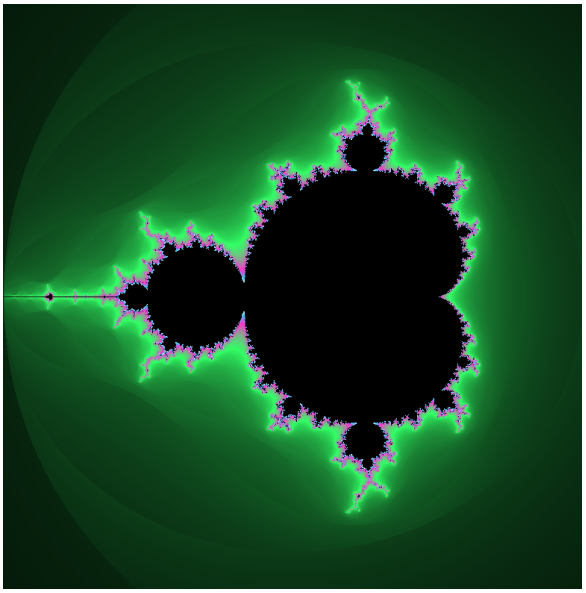
(a) Preset classic.



(b) Preset fire.



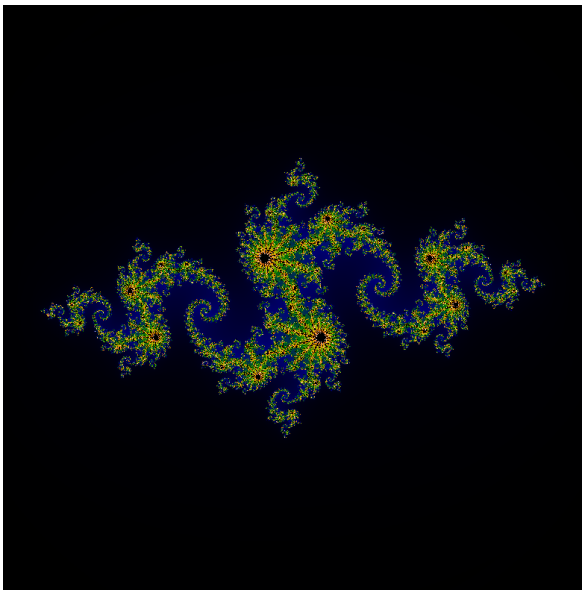
(c) Preset ice.



(d) Preset neon.

Rysunek 6.2: Wygenerowane obrazy zbioru Mandelbrota (50 iteracji) wykorzystujące predefiniowane palety kolorów.

36:4760612645



(a) Fraktal Julii w presecie classic.



(b) Fraktal Julii w trybie niestandardowym.

Rysunek 6.3: Porównanie schematów kolorystycznych fraktala Julii (300 iteracji) — w trybie niestandardowym zostały zmienione kolory i liczba przedziałów chromatycznych oraz wartość gamma.

37:3464425811

Rozdział 7

Animacja i eksport

Moduł animacji rozszerza interaktywną eksplorację o możliwość utrwalenia ruchu kamery w postaci pliku wideo. Rozdział opisuje przyjęty model klatek kluczowych, sposób wyznaczania trajektorii kamery między nimi oraz dostępne formaty wyjściowe.

7.1. Animacja parametrów, klatki, przebieg animacji

Moduł animacji pozwala na generowanie wideo przedstawiającego ruch kamery po obrazie fraktala. Podstawowym pojęciem jest *klatka kluczowa* (ang. *keyframe*) — zapis stanu kamery w określonej chwili, obejmujący pozycję, skalę i wymiary obrazu. Użytkownik rejestruje klatki kluczowe ręcznie: nawiguje kamerę do interesującej pozycji i naciska przycisk „add keyframe". Przycisk wyświetla aktualną liczbę zarejestrowanych klatek. Klatki można usunąć przyciskiem „clear keyframes" (operacja usuwa wszystkie klatki naraz).

Do wygenerowania animacji wymagane są co najmniej dwie klatki kluczowe. Algorytm interpolacji wyznacza pośrednie stany kamery pomiędzy kolejnymi parami klatek, generując płynny ruch. Dla każdego interpolowanego stanu kamery renderowana jest pełna klatka fraktala z aktualnym schematem kolorowania.

Częstotliwość klatek animacji (ang. *frame rate*) jest konfigurowalna w zakresie 1–60 FPS.

7.2. Interpolacja ruchu

Dla każdej pary sąsiednich klatek kluczowych parametr interpolacji $t \in [0, 1]$ przyjmuje wartości od 0 (klatka początkowa segmentu) do 1 (klatka końcowa). W obu przypadkach pełna trajektoria animacji powstaje poprzez sklejenie segmentów dla wszystkich par klatek.

Zaimplementowane zostały dwa tryby interpolacji, porównane później na rysunku 7.1.

Interpolacja liniowa. Między każdą parą sąsiednich klatek kluczowych parametry kamery są interpolowane liniowo w czasie. Skala kamery jest interpolowana w przestrzeni logarytmicznej:

$$\log s(t) = (1 - t)\ \log s_1 + t\ \log s_2 \tag{7.1}$$

co odpowiada wykładniczej zmianie skali — jednorodnej wizualnie przy zoomie. Położenie kamery jest interpolowane proporcjonalnie do skali, a nie liniowo w czasie, co zapewnia optycznie prostą trajektorię ruchu podczas jednoczesnego przybliżania.

Interpolacja krzywą Catmull-Rom. Krzywa Catmull-Rom [1, s. 317] jest rodzajem interpolującego splajnu sześciennego, który przechodzi dokładnie przez wszystkie klatki kluczowe (w odróżnieniu od splajnów aproksymujących). Dla każdego segmentu między klatkami p_1 i p_2

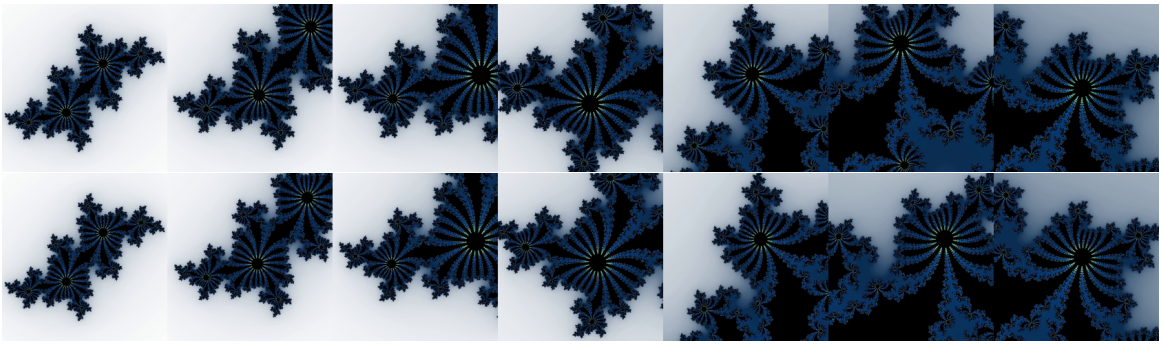
38:1388662731

uwzględniane są sąsiednie klatki p_0 i p_3, co pozwala na gładkie przejście z ciągłymi pierwszymi pochodnymi w każdym punkcie kontrolnym. Wzór dla jednego wymiaru:

$$s(t) = \tfrac{1}{2}\Big[2p_1 + (p_2 - p_0)\,t + (2p_0 - 5p_1 + 4p_2 - p_3)\,t^2 + (3p_1 - p_0 - 3p_2 + p_3)\,t^3\Big] \tag{7.2}$$

Na brzegach sekwencji klatek kluczowych brakujące punkty fantomowe p_0 i p_3 są wyznaczane przez odbicie symetryczne istniejących klatek, co zapobiega oscylacjom na początku i końcu animacji. Skala kamery jest interpolowana w przestrzeni logarytmicznej analogicznie jak w trybie liniowym.

Zauważmy, że dla zaledwie dwóch klatek kluczowych interpolacja krzywą Catmull-Rom redukuje się dokładnie do interpolacji liniowej w przestrzeni współrzędnych. Wynika to z odbicia symetrycznego punktów fantomowych: po podstawieniu $p_0 = 2p_1 - p_2$ oraz $p_3 = 2p_2 - p_1$ współczynniki przy t^2 oraz t^3 we wzorze 7.2 zerują się tożsamościowo. Wizualne różnice między trybami interpolacji są zatem widoczne dopiero przy trzech lub większej liczbie klatek kluczowych.



Rysunek 7.1: Porównanie graficzne działania interpolacji czterech punktów kluczowych: na górze przedstawiona jest interpolacja liniowa, na dole interpolacja krzywą Catmull-Rom.

7.3. Format i opcje eksportu

Po wygenerowaniu animacji użytkownik wybiera format wyjściowy:

— **GIF** — format bezstratny, obsługuje animacje, szeroko kompatybilny. Ograniczenie do 256 kolorów na klatkę może prowadzić do widocznych pasmowości przy subtelnych gradientach. Zapis realizowany przez Pillow.
— **MP4** — format stratny, kodek H.264 (`libx264`). Najlepszy stosunek jakości do rozmiaru pliku; zalecany do dłuższych animacji. Zapis realizowany przez imageio z backendem FFmpeg.
— **WebM** — format stratny, kodek VP9 (`libvpx-vp9`), ze stałą jakością (`-crf 30`). Dobra jakość przy małych plikach; natywnie obsługiwany przez przeglądarki internetowe. Zapis przez imageio.

Pliki eksportowane są do podkatalogów `animations/gif/`, `animations/mp4/` i `animations/webm/`, tworzonych automatycznie przy starcie aplikacji. Nazwy plików zawierają znacznik czasu w formacie `YYYY-MM-DD_HH-MM-SS`. Po zakończeniu eksportu obok przycisku „export animation” wyświetlany jest komunikat z nazwą zapisanego pliku.

Rozdział 8

Wyniki i analiza działania programu

Rozdział przedstawia ocenę zrealizowanej aplikacji przez pryzmat poprawności generacji, testu limitów i wydajności działania. Omawia interesujące obszary eksploracji fraktali.

8.1. Poprawność generacji fraktali

Weryfikacja poprawności implementacji algorytmów fraktalnych nie poddaje się prostej kontroli „piksel po pikselu". Granica większości zaimplementowanych zbiorów ma strukturę fraktalną, a różnice subpikselowe, kolejność operacji zmiennoprzecinkowych czy drobne zmiany w schemacie kolorowania prowadzą do nieidentycznych obrazów dwóch poprawnych implementacji.

Zamiast porównywania całych klatek przyjęto podejście oparte na zbiorze *punktów charakterystycznych*: wartości parametru c, dla których przynależność do zbioru można wyprowadzić analitycznie. Punkty te dzielą się na trzy grupy: wewnętrzne (orbita ograniczona), zewnętrzne (orbita ucieka) oraz graniczne (leżące na brzegu zbioru).

Punkty wewnętrzne. Wybrano trzy wartości. Dla $c = 0$ iteracja $f_0(z) = z^2$ wystartowana z zera daje $z_n = 0$ dla każdego n — orbita stoi w miejscu i nigdy nie ucieka. Dla $c = -1$ ciąg iteracji ma postać $0 \to -1 \to 0 \to -1 \to \ldots$ — orbita wpada w cykl o długości 2. Dla $c = -0{,}5$ orbita zbiega do punktu $z^* \approx -0{,}366$, który jest punktem stałym przyciągającym (każda kolejna iteracja zbliża się do niego), a więc również pozostaje ograniczona.

Punkty zewnętrzne. Dobrano $c = 3$ (każdy punkt o $|c| > 2$ ucieka natychmiast), $c = -2{,}5$ (orbita ucieka już po pierwszym kroku) oraz $c = 0{,}5 + 0{,}5i$ (orbita ucieka po kilku iteracjach). Każdy z punktów zweryfikowano przez ręczne obliczenie kilku pierwszych wyrazów ciągu iteracji.

Punkty graniczne. W teście uwzględniono dwa punkty leżące dokładnie na brzegu zbioru: $c = 0{,}25$ — czubek głównej kardioidy oraz $c = -0{,}75$ — punkt rozwidlenia, w którym zbiór przechodzi z obszaru o jednym cyklu do obszaru o dwóch cyklach. Brzeg zbioru jest matematycznie „nieskończenie cienki", więc żaden algorytm pracujący na liczbach o skończonej precyzji nie rozstrzygnie przynależności tych punktów jednoznacznie — wynik zależy od przyjętej konwencji. W zaimplementowanej wersji takie punkty pozostają w granicy iteracji i są klasyfikowane jako wewnętrzne.

Wyniki testów. Tabela 8.1 przedstawia wyniki testu dla $N_{\max} = 500$ i progu ucieczki $R = 2$. Wartość równa $N_{\max}$ oznacza klasyfikację punktu jako wewnętrznego. Test przeprowadzono niezależnie na trzech maszynach testowych — otrzymane wartości były identyczne w granicach precyzji `float64`.

40:6505696331

Tabela 8.1: Wyniki testów punktów charakterystycznych dla zbioru Mandelbrota.

Punkt c	Charakterystyka	Wynik	Oczekiwane
0	punkt stały $z = 0$	500,00	wewnątrz
-1	orbita o okresie 2	500,00	wewnątrz
$-0{,}5$	wnętrze głównej kardioidy	500,00	wewnątrz
3	$\lvert c\rvert > 2$, natychmiastowa ucieczka	0,86	na zewnątrz
$0{,}5 + 0{,}5i$	ucieczka po kilku krokach	4,66	na zewnątrz
$-2{,}5$	ucieczka po pierwszym kroku	1,13	na zewnątrz
0,25	ostrze głównej kardioidy	500,00	granica
$-0{,}75$	punkt rozwidlenia okresu	500,00	granica

Wszystkie sześć punktów rozstrzygalnych zostało sklasyfikowanych poprawnie, a punkty graniczne zachowują się zgodnie z przyjętą konwencją.

Weryfikacja krzyżowa z istniejącymi narzędziami. Niezależnie od testu punktowego porównano obraz wynikowy z obrazami generowanymi przez XaoS i Kalles Fraktaler dla tego samego kadrowania i tej samej liczby iteracji. Pomijając różnice wynikające ze schematów kolorowania, struktura granicy zbioru oraz kształt głównych miniatur były jakościowo zgodne. Analogiczny test przeprowadzono dla wybranych zbiorów Julii ($c = -0{,}8+0{,}156i$ i $c = 0{,}285+0{,}01i$) oraz fraktala Burning Ship; nie zaobserwowano rozbieżności strukturalnych.

Paproć Barnsleya — test prostokąta ograniczającego. Empirycznie ustalone granice atraktora paproci Barnsleya pod standardowymi regułami afinicznymi to $x \in [2{,}1820; 2{,}6558]$ oraz $y \in [0; 9{,}9984]$. Każdy błąd znaku lub permutacja współczynników w którymkolwiek z czterech przekształceń $T_1, \ldots, T_4$ wypchnęłaby część generowanych punktów poza ten obszar. Test automatyczny sprawdza, że dla $N = 200{,}000$ i ziarna 42 wszystkie wygenerowane punkty mieszczą się w zakładanych granicach z tolerancją 10^{-3}, co potwierdza poprawność zaimplementowanych współczynników z tabeli 2.2.

Trójkąt Sierpińskiego — test rekurencji wokół wierzchołków. Każdy z trzech wierzchołków v_0, v_1, v_2 trójkąta jest punktem stałym jednego z przekształceń chaos game o współczynniku kontrakcji równym $\frac{1}{2}$. Z definicji algorytmu wynika, że orbita musi gęsto odwiedzać otoczenia wszystkich wierzchołków — szansa wybrania danej reguły k razy z rzędu wynosi $\frac{1}{3}^k$, co po ciągu aplikacji sprawia, że punkt znajduje się w odległości $\leqslant \frac{1}{2}^k \cdot d_0$ od odpowiedniego wierzchołka. Test automatyczny sprawdza, że dla $N = 100{,}000$ punktów i ziarna 42 minimalna odległość chmury od każdego wierzchołka jest mniejsza niż 10^{-2} — oczekiwana asymptotyczna wartość rzędu $\frac{1}{2}^{\log_3(N)} \approx 7{,}004 \cdot 10^{-4}$ daje znaczący zapas wobec zadanego progu.

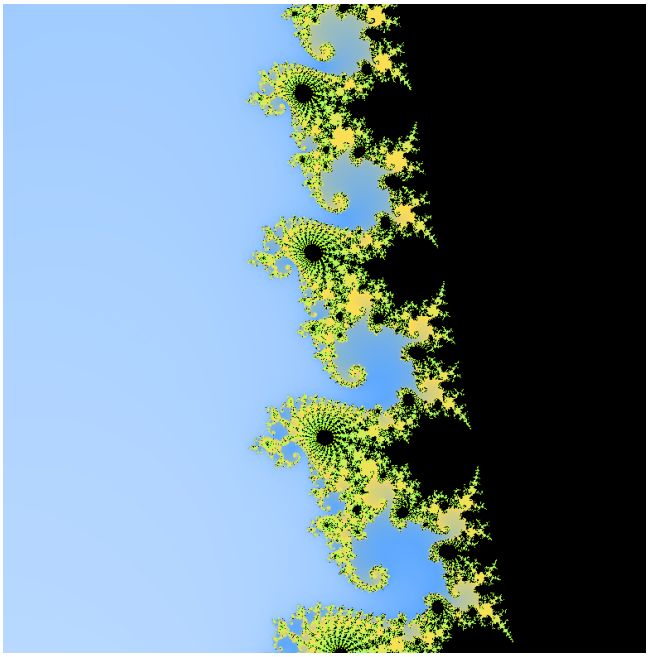
8.2. Ciekawe obszary i obserwacje wizualne

Interaktywna eksploracja zaimplementowanych fraktali pozwala na bezpośrednie obserwowanie własności matematycznych opisanych w rozdziale 1 — samopodobieństwa, wieloskalowości oraz złożoności granicy zbioru. W tym podrozdziale przedstawiono wybrane obszary szczególnie interesujące pod względem wizualnym oraz poznawczym, wraz z krótkim komentarzem wyjaśniającym ich strukturę.

41:2569583012

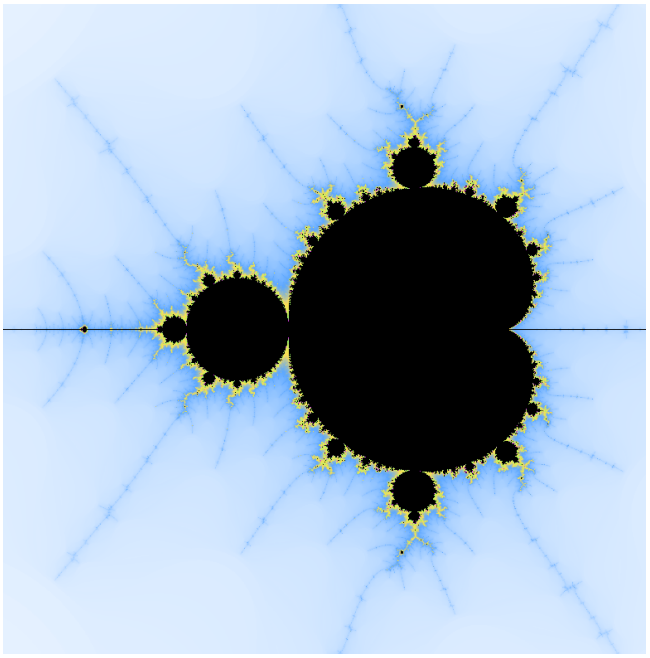
8.2.1. Zbiór Mandelbrota

Domyślny widok zbioru Mandelbrota pokazuje całą strukturę zbioru — charakterystyczną kardioidę z przyległym okręgiem oraz mniejsze „wypustki". Ciekawsze obszary ujawniają się dopiero przy stopniowym przybliżaniu do granicy.



Rysunek 8.1: Dolina koników morskich

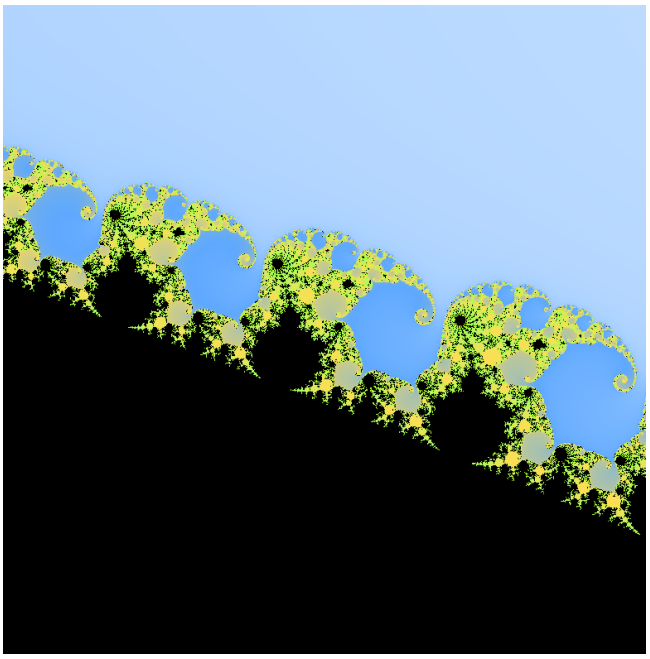
Obszar w pobliżu $c \approx -0{,}745 + 0{,}113i$, znany jako „dolina koników morskich". Widoczne są charakterystyczne, spiralne struktury przypominające koniki morskie, ułożone wzdłuż wąskich pasm między dwoma większymi obszarami zbioru.



Rysunek 8.2: Miniaturowy Mandelbrot

42:1116701009

Obszar w okolicy $c \approx -1{,}75 + 0i$. Przy odpowiednim zoomie ujawnia się kompletna, miniaturowa kopia całego zbioru Mandelbrota, otoczona strukturami ozdobnymi. Obserwacja ta empirycznie ilustruje samopodobieństwo zbioru: cały zbiór odtwarza swoją strukturę na każdym poziomie skali.



Rysunek 8.3: Dolina słoni

Obszar w okolicy $c \approx 0{,}275 + 0i$. Charakteryzuje się gęstymi, regularnie ułożonymi spiralami i strukturami przypominającymi grzbiety słoni, z widoczną trąbą.

8.2.2. Zbiory Julii

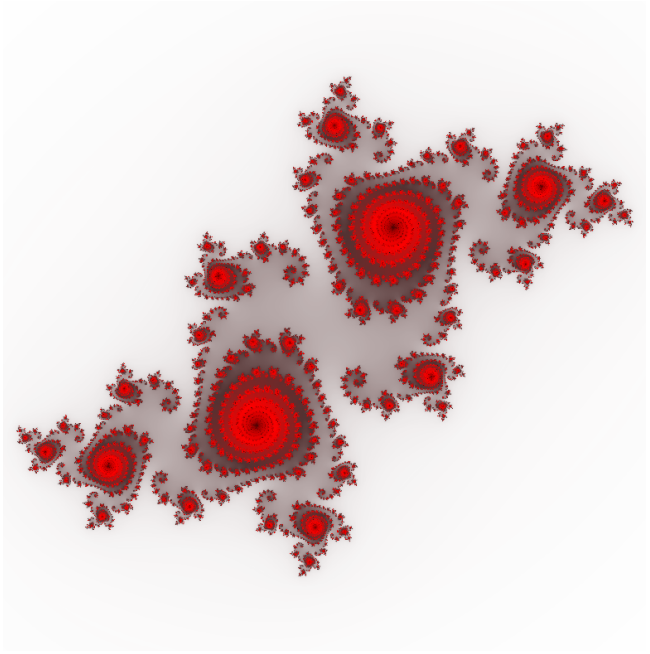
Wartość parametru c ma decydujący wpływ na kształt zbioru Julii. Wartości leżące wewnątrz zbioru Mandelbrota dają zbiory spójne, wartości zewnętrzne — zbiory zdezintegrowane.

43:1657017923



Rysunek 8.4: Dolina krzyku

Parametr $c = -0{,}755 + 0{,}049i$, współrzędne $(0{,}430, -0{,}08)$ przedstawia dolinę krzyku, najlepiej widoczną, gdy $N_{\max} = 300$. Elementy doliny zawijają się w podobne do ludzkiej twarzy kształty, z widocznymi szeroko otwartymi ustami oraz dwoma wklęsłościami przypominającymi oczy.



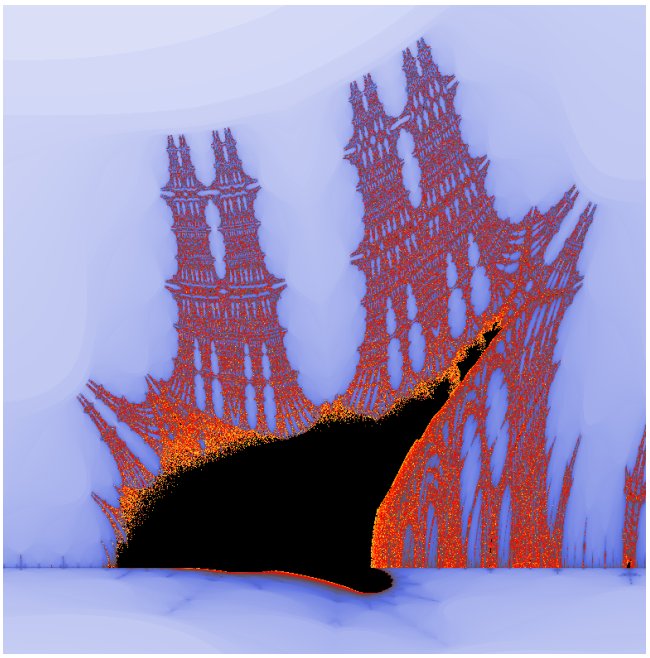
Rysunek 8.5: Julia poza zbiorem Mandelbrota

Parametr $c = -0{,}197 + 0{,}656i$ leży poza zbiorem Mandelbrota, w bezpośrednim sąsiedztwie jego granicy, przez co zbiór nie jest spójny. Składa się z izolowanych „wysp" o spiralnej budowie.

44:6048505392

8.2.3. Burning Ship

Burning Ship wyróżnia się charakterystycznym, asymetrycznym kształtem głównej struktury, przypominającym płonący okręt — skąd pochodzi nazwa fraktala [6].



Rysunek 8.6: Płonące maszty

Obszar w okolicy $c \approx -1{,}755 - 0{,}03i$. Charakterystyczna, długa pionowa struktura o subtelnie powtarzającym się wzorze.

8.3. Wydajność, czas renderowania, testy limitów

Podrozdział przedstawia pomiary czasu renderowania pojedynczej klatki dla wszystkich zaimplementowanych algorytmów. Pomiary mają na celu ilościową ocenę zachowania aplikacji w typowych scenariuszach użycia oraz zidentyfikowanie czynników o największym wpływie na wydajność.

8.3.1. Środowisko testowe

Testy przeprowadzono na trzech maszynach reprezentujących różne klasy sprzętu konsumenckiego: stacjonarnym komputerze klasy „desktop”, starszym laptopie ultramobilnym oraz nowoczesnym laptopie z procesorem opartym na architekturze ARM. Specyfikację każdej z maszyn zestawiono w tabeli 8.2.

We wszystkich przypadkach środowisko uruchomieniowe stanowił interpreter CPython w wersji 3.12 z bibliotekami NumPy 2.3.5 oraz Numba 0.62.1. Aplikacja działała z aktywnym, ale niewykorzystywanym podczas pomiaru, modułem GUI; sam pomiar dotyczył wyłącznie czasu wykonania funkcji generującej siatkę wartości iteracyjnych (`generate_frame`), bez kosztu nakładania palety barwnej i aktualizacji tekstury.

45:4264464185

Tabela 8.2: Specyfikacja sprzętowa stanowisk testowych.

	Komputer (PC1)	Laptop Win (PC2)	Laptop Mac (PC3)
Procesor	AMD Ryzen 7 3800X	Intel Core i7-10510U	Apple M4 Max
Architektura	x86_64 (Zen 2)	x86_64 (Comet Lake)	ARM64 (Apple silicon)
System	Windows 11 Home	Windows 11 Home	macOS 26.4.1 Tahoe

8.3.2. Metodologia pomiarów

Czas renderowania pojedynczej klatki mierzony był jako średnia z pięciu kolejnych wywołań tej samej funkcji, poprzedzonych jednym wywołaniem „rozgrzewającym", którego celem było wyeliminowanie z pomiarów kosztu kompilacji JIT. Funkcje pomiarowe nie korzystały z cache wyników IFS (`lru_cache` został wyłączony), tak aby każdy pomiar odpowiadał rzeczywistemu kosztowi obliczeń. Mierzono trzy parametry kontrolne: rozdzielczość obrazu, maksymalną liczbę iteracji ($N_{\max}$ dla fraktali escape-time) oraz liczbę punktów (N dla fraktali IFS). Wartości progowe ucieczki, parametry kamery oraz ziarno generatora liczb pseudolosowych były we wszystkich pomiarach takie same.

8.3.3. Odchylenie standardowe

Dla każdego punktu pomiarowego rejestrowano również odchylenie standardowe próby. Dla fraktali escape-time obserwowane względne odchylenie standardowe nie przekraczało 3% wartości średniej w żadnym z testów; dla fraktali IFS było wyższe i sięgało 5–7% przy najmniejszych liczbach punktów, spadając do około 2% dla większych N. Wartości te uzasadniają traktowanie różnic poniżej około 5–10% między poszczególnymi konfiguracjami jako statystycznie nieistotnych. Z tego względu w analizie skupiono się na trendach skalowania (zachowanie czasu w funkcji $N_{\max}$ i rozdzielczości), a nie na pojedynczych wartościach bezwzględnych.

8.3.4. Wyniki dla fraktali escape-time

Tabele 8.3–8.5 przedstawiają wyniki pomiarów dla zbioru Mandelbrota, zbioru Julii (z parametrem $c = -0{,}8 + 0{,}156i$) oraz fraktala Burning Ship. Wartości w tabelach podano w milisekundach.

Tabela 8.3: Średni czas generowania klatki zbioru Mandelbrota [ms].

	PC1 (Ryzen 3800X)			PC2 (i7-10510U)			PC3 (M4 Max)		
$N_{\max}$	100	300	1000	100	300	1000	100	300	1000
800×600	31,1	59,8	158,5	39,0	82,5	229,8	17,0	45,0	142,1
1200×800	58,2	109,1	283,4	75,1	155,5	419,2	30,6	81,0	250,1
1920×1080	115,8	210,3	526,9	149,8	284,5	740,7	57,4	149,6	461,5

Skalowanie z rozdzielczością. W każdym z trzech fraktali escape-time czas renderowania rośnie niemal liniowo wraz z liczbą pikseli. Przejście z rozdzielczości 800×600 (480 000 pikseli) na 1920×1080 (2 073 600 pikseli), czyli wzrost o czynnik około 4,3, prowadzi do wzrostu czasu

46:4555080537

Tabela 8.4: Średni czas generowania klatki zbioru Julii [ms].

	PC1 (Ryzen 3800X)			PC2 (i7-10510U)			PC3 (M4 Max)		
$N_{\max}$	100	300	1000	100	300	1000	100	300	1000
800×600	27,0	36,3	40,0	33,2	45,9	51,3	14,3	23,1	26,9
1200×800	51,3	68,3	75,6	70,9	97,0	108,1	25,8	42,0	48,6
1920×1080	104,5	133,4	144,9	164,3	216,6	234,3	49,9	79,6	90,5

Tabela 8.5: Średni czas generowania klatki fraktala Burning Ship [ms].

	PC1 (Ryzen 3800X)			PC2 (i7-10510U)			PC3 (M4 Max)		
$N_{\max}$	100	300	1000	100	300	1000	100	300	1000
800×600	35,3	71,3	193,8	55,8	109,1	307,9	22,4	61,2	190,2
1200×800	65,4	129,1	345,5	103,6	238,1	486,1	41,0	108,6	340,9
1920×1080	130,4	245,8	639,3	152,7	319,6	875,7	76,2	200,0	619,8

o czynnik 3,3–4,5 niezależnie od użytego sprzętu i fraktala. Nieznaczne odchylenia od skalowania liniowego można przypisać kosztom alokacji pamięci, efektom buforowania linii pamięci podręcznej oraz zmienności obciążenia tła systemu operacyjnego.

Skalowanie z $N_{\max}$ — różnica między Mandelbrot/Burning Ship a Julia. Jednym z najbardziej wyrazistych wyników jest jakościowa różnica w skalowaniu czasu względem maksymalnej liczby iteracji pomiędzy Mandelbrotem (oraz Burning Ship) a zbiorem Julii. Dla rozdzielczości 1200×800 na PC1 czas Mandelbrota rośnie z 58,2 ms ($N_{\max} = 100$) do 283,4 ms ($N_{\max} = 1000$), czyli niemal pięciokrotnie przy dziesięciokrotnym wzroście parametru. W tych samych warunkach czas zbioru Julii wzrasta z 51,3 ms zaledwie do 75,6 ms, czyli mniej niż o połowę.

Przyczyną tej różnicy jest ilość pikseli w kadrze należących do zbioru. Piksele wewnątrz zbioru zawsze wykonują pełne $N_{\max}$ iteracji; piksele poza zbiorem uciekają zwykle po kilkunastu krokach i są praktycznie niewrażliwe na wartość $N_{\max}$. Domyślny kadr Mandelbrota obejmuje dużą część zbioru, więc czas rośnie niemal liniowo z $N_{\max}$. W zbiorze Julii dla $c = -0{,}8 + 0{,}156i$ atraktor jest cienki i fragmentaryczny — pikseli wewnętrznych jest niewiele, więc czas jest mniej zależny od limitu iteracji. Burning Ship zachowuje się podobnie do Mandelbrota, jest jednak o około 20% wolniejszy ze względu na dodatkowe operacje wartości bezwzględnej.

Porównanie międzyplatformowe. Najszybszą platformą okazał się laptop wyposażony w procesor Apple M4 Max — dla większości testów osiągał on czasy 1,3–2,1 raza krótsze niż PC1 z procesorem Ryzen 3800X. Najwolniejszą maszyną był laptop ultramobilny PC2, którego czasy były około 30–40% gorsze niż na PC1. Co ciekawe, przewaga procesora M4 Max nad pozostałymi maszynami jest wyraźnie większa dla fraktali punktowych (patrz podrozdział 8.3.5) niż dla escape-time. W przypadku Mandelbrota przy 1200×800 i $N_{\max} = 300$ M4 Max osiąga 81,0 ms wobec 109,1 ms na Ryzenie (∼1,35× szybciej), podczas gdy dla 1 000 000 punktów paproci Barnsleya otrzymano odpowiednio 14,8 ms i 46,5 ms (∼3,1× szybciej). Różnica ta sugeruje, że jądro escape-time jest silnie ograniczone arytmetyką i pamięcią podręczną na każdej z platform, podczas gdy prosty kod IFS lepiej wykorzystuje wysoką wydajność pojedynczego rdzenia M4 oraz wydajność generatora liczb pseudolosowych.

47:2643345931

8.3.5. Wyniki dla fraktali IFS

Tabela 8.6 zawiera czasy generowania chmury punktów i jej rasteryzacji dla trójkąta Sierpińskiego oraz paproci Barnsleya przy czterech wartościach parametru N (liczba punktów).

Tabela 8.6: Średni czas generowania klatki fraktali IFS [ms].

Liczba punktów N	PC1 (Ryzen 3800X)	PC2 (i7-10510U)	PC3 (M4 Max)
Trójkąt Sierpińskiego			
10 000	1,1	1,3	0,2
100 000	3,3	4,4	1,1
1 000 000	39,9	46,5	10,9
5 000 000	245,2	220,0	55,4
Paproć Barnsleya			
10 000	1,3	1,6	0,2
100 000	4,8	6,4	1,5
1 000 000	46,5	55,5	14,8
5 000 000	249,7	277,8	72,7

Skalowanie czasu względem liczby punktów jest niemal idealnie liniowe: zwiększenie N o czynnik 10 prowadzi do wzrostu czasu o czynnik 9,5–12,5 we wszystkich przypadkach. Potwierdza to wyprowadzoną w rozdziale 2 złożoność obliczeniową $O(N)$ algorytmu chaos game. Paproć Barnsleya jest konsekwentnie nieco wolniejsza od trójkąta Sierpińskiego, co wynika z większej liczby przekształceń afinicznych (cztery zamiast trzech) oraz konieczności obliczania skumulowanego rozkładu prawdopodobieństwa przed wyborem przekształcenia.

8.3.6. Koszt kompilacji JIT

Numba kompiluje funkcje opatrzone dekoratorem `@jit(nopython=True)` przy pierwszym wywołaniu, co wprowadza jednorazowy narzut czasowy nieobecny w kolejnych wywołaniach. Tabela 8.7 przedstawia wynik dedykowanego pomiaru wykonanego w świeżym podprocesie (aby zapewnić, że stan kompilatora Numby był pusty), dla zbioru Mandelbrota o rozdzielczości 1200×800 i $N_{\max} = 300$.

Tabela 8.7: Koszt kompilacji JIT (Mandelbrot, 1200×800, $N_{\max} = 300$).

	PC1	PC2	PC3
Pierwsze wywołanie [ms]	562	712	246
Kolejne wywołanie [ms]	110,9	169,8	80,7
Narzut kompilacji [ms]	451	542	165

Narzut kompilacji wynosi od 165 ms (PC3) do 542 ms (PC2). W kontekście ogólnego czasu pracy aplikacji jest on pomijalny — ponoszony jest jeden raz na typ fraktala w trakcie sesji — jednak jest wyraźnie odczuwalny przy pierwszym przełączeniu na każdy z algorytmów, manifestując się jako jednorazowe „zacięcie” obrazu trwające ułamek sekundy. Aby ograniczyć ten efekt w postrzeganiu użytkownika, najczęściej używany algorytm (zbiór Mandelbrota) jest

48:7153493494

wybierany domyślnie przy starcie aplikacji, co pozwala na wczesne wywołanie kompilacji bez interakcji ze strony użytkownika.

8.3.7. Wnioski wydajnościowe

Pomiary potwierdzają teoretyczne złożoności obliczeniowe wszystkich zaimplementowanych algorytmów oraz pokazują, że aplikacja zachowuje interaktywność w typowych warunkach użycia (rozdzielczość rzędu 1200×800, $N_{\max} = 100$–300, $N \approx 10^6$ dla IFS) nawet na sprzęcie kilkuletnim. Dla bardziej wymagających ustawień — wysokiej rozdzielczości 1920×1080 z $N_{\max} = 1000$, lub chmury 5 000 000 punktów — czas renderowania pojedynczej klatki sięga 0,5–1 s, co czyni aktywną nawigację bez mechanizmu adaptacyjnego obniżania jakości (omawianego w podrozdziale 8.4) niewykonalną. Zachowanie płynnej eksploracji w takich warunkach jest możliwe wyłącznie dzięki opisanemu w rozdziale 4 obniżaniu rozdzielczości i $N_{\max}$ podczas ruchu kamery.

8.4. Ograniczenia przybliżania i kompromisy jakości

Interaktywna eksploracja fraktali stawia przed implementacją dwa typy ograniczeń: matematyczne, wynikające ze skończonej precyzji arytmetyki, oraz praktyczne, związane z czasem obliczeń i koniecznością utrzymania płynności interfejsu.

8.4.1. Ograniczenie precyzji arytmetyki `float64`

Wszystkie obliczenia w aplikacji są prowadzone w 64-bitowej arytmetyce zmiennoprzecinkowej standardu IEEE 754, oferującej około 15–17 cyfr znaczących mantysy. Ograniczenie to przekłada się bezpośrednio na maksymalny dopuszczalny stopień przybliżenia obrazu fraktala.

Domyślny kadr zbioru Mandelbrota obejmuje obszar płaszczyzny zespolonej o szerokości około 3,5 jednostki. Przy zoomie rzędu 10^{13} szerokość pola widzenia wynosi około $3{,}5 \cdot 10^{-13}$ — wartość bliską najmniejszemu przyrostowi reprezentowalnemu w `float64` dla liczb rzędu jedności. W praktyce różnica między współrzędnymi sąsiednich pikseli staje się porównywalna z precyzją reprezentacji zmiennoprzecinkowej, a w skrajnym przypadku przestaje być w niej w ogóle reprezentowalna.

Manifestacja artefaktów. Skutki ograniczenia pojawiają się stopniowo wraz ze wzrostem zoomu. Do współczynnika rzędu 10^{12} obraz wygląda poprawnie. Powyżej 10^{13} pojawiają się widoczne „bloki” — prostokątne obszary pikseli o identycznej barwie, którym przypisano dokładnie tę samą wartość zmiennoprzecinkową współrzędnej. Powyżej 10^{14} obraz przybiera charakter mozaiki, w której struktura fraktalna staje się nieczytelna; algorytm zwraca tę samą liczbę iteracji dla wielu sąsiadujących pikseli, ponieważ ich punkty startowe są w arytmetyce `float64` nieodróżnialne. Oba omówione przypadki przedstawione zostały na figurze 8.7.

W praktyce aplikacja pozwala bez zauważalnych artefaktów eksplorować zbiór Mandelbrota do współczynnika zoomu rzędu 10^{12}, co dla większości zastosowań edukacyjnych i artystycznych jest w zupełności wystarczające. Głębsza eksploracja, porównywalna z możliwościami wyspecjalizowanych narzędzi takich jak Kalles Fraktaler (zoom rzędu 10^{1000} i więcej) wymagałaby zmiany arytmetyki podstawowej.

49:7090653188



(a) Powiększenie rzędu 10^{13} fraktala Mandelbrota.



(b) Powiększenie rzędu 10^{14} fraktala Mandelbrota.

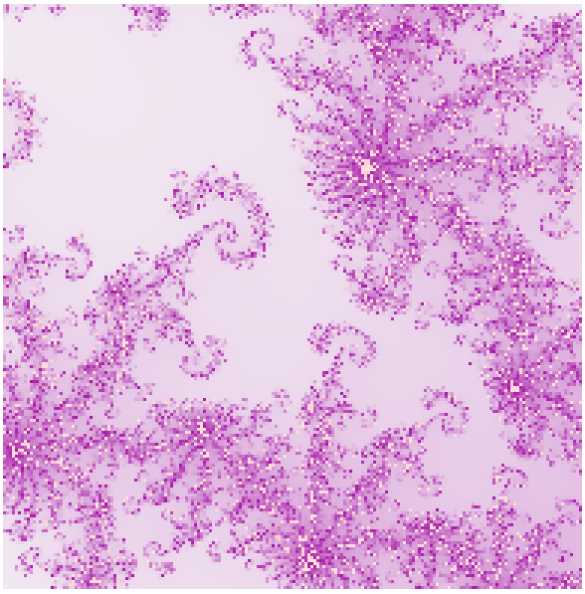
Rysunek 8.7: Zobrazowanie limitów precyzji powiększania przy arytmetyce `float64`.

8.4.2. Adaptacyjne obniżanie jakości podczas nawigacji

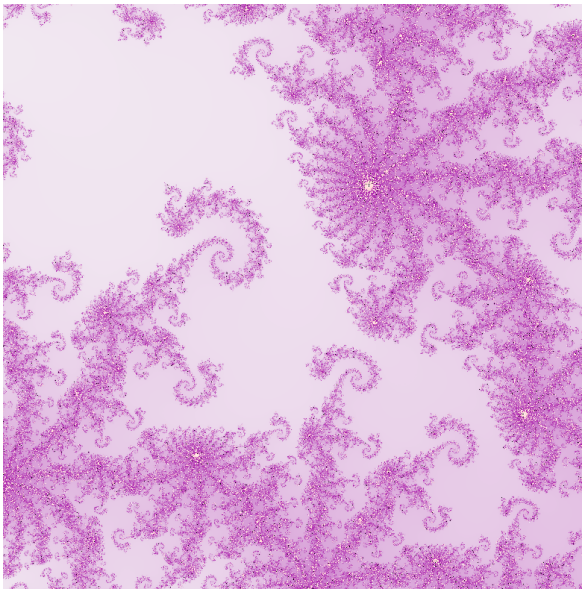
Drugą klasą jest czas obliczeń pojedynczej klatki. Jak pokazano w podrozdziale 8.3, może on sięgać setek milisekund dla wymagających ustawień, podczas gdy płynna nawigacja wymaga aktualizacji obrazu z częstotliwością 20–60 Hz (budżet czasowy 17–50 ms na klatkę). Bezpośrednie renderowanie w pełnej jakości podczas ruchu kamery prowadziłoby do widocznego „zacinania się" obrazu.

Aplikacja stosuje mechanizm adaptacyjnego obniżania jakości opisany szczegółowo w rozdziale 4: w trakcie ruchu generowana jest klatka o zmniejszonej rozdzielczości i mniejszej liczbie iteracji, skalowana następnie do pełnych wymiarów metodą najbliższego sąsiada. Po zakończeniu ruchu natychmiast renderowana jest klatka w pełnej jakości, zastępująca obraz tymczasowy.

Dobór parametrów. Domyślne wartości to dwukrotne zmniejszenie liniowych wymiarów obrazu (szesnastokrotne zmniejszenie liczby pikseli) oraz ograniczenie $N_{\max}$ do 300. Łącznie daje to przyspieszenie od 30 do 60 razy, wystarczające dla utrzymania płynnej nawigacji w bardzo wymagających scenariuszach. Użytkownik może modyfikować te parametry w panelu kamery: współczynnik zmniejszenia rozdzielczości (*downscale factor*) w zakresie 2–16 oraz limit iteracji (*iteration limit*) w zakresie 1–1000. Mechanizm można również całkowicie wyłączyć.



(a) 4–krotne zbliżenie jakości ruchu, limit iteracji 500.



(b) Czysta generacja kawałka fraktala Julii (1000 iteracji).

Rysunek 8.8: Graficzne porównanie adaptacyjnego obniżania jakości z wygenerowanym oryginałem części fraktala Julii przy ustawieniu maksymalnej iteracji programu.

51:1115953458

Rozdział 9

Porównanie z istniejącymi rozwiązaniami

Rozdział zestawia FractalSurfer z trzema istniejącymi narzędziami, reprezentującymi różne podejścia do wizualizacji fraktali: XaoS, Ultra Fractal i Kalles Fraktaler. Wybór obejmuje narzędzie open source o szerokim zakresie fraktali (XaoS), pakiet komercyjny dla zaawansowanych użytkowników (Ultra Fractal) oraz program wyspecjalizowany w głębokim zoomie (Kalles Fraktaler). Porównanie obejmuje zakres obsługiwanych fraktali, sposób renderowania, precyzję obliczeń, dostępność animacji oraz dystrybucję.

9.1. Przegląd programów dostępnych w internecie

XaoS. XaoS jest otwartoźródłowym, wieloplatformowym programem do interaktywnej eksploracji fraktali, utrzymywanym aktywnie od lat dziewięćdziesiątych XX wieku [15]. Napisany w języku C, oferuje bardzo wysoką wydajność renderowania poprzez zastosowanie algorytmu przyrostowego (ang. *fast zoom*), który zamiast przeliczać cały obraz od nowa, ponownie wykorzystuje piksele z poprzedniej klatki. Dzięki temu nawigacja w XaoS jest płynna nawet na starszym sprzęcie. Program obsługuje kilkanaście typów fraktali, posiada edytor palet barwnych oraz tryb automatycznego odtwarzania animacji. XaoS nie oferuje jednak modułu animacji opartego na klatkach kluczowych ani eksportu do standardowych formatów wideo.

Ultra Fractal. Ultra Fractal jest komercyjnym programem dla systemów Windows i macOS, skierowanym do zaawansowanych użytkowników i artystów cyfrowych [16]. Wyróżnia się edytorem formuł umożliwiającym definiowanie własnych funkcji iteracji oraz systemem warstw pozwalającym łączyć kilka fraktali w jednym obrazie. Posiada rozbudowany moduł animacji z klatkami kluczowymi i eksportem do wideo. Ultra Fractal jest narzędziem znacznie wykraczającym poza zakres edukacyjny — jego złożoność i model licencyjny czynią go rozwiązaniem profesjonalnym, nieprzeznaczonym do szybkiej, swobodnej eksploracji.

Kalles Fraktaler. Kalles Fraktaler jest programem dla systemu Windows wyspecjalizowanym w ekstremalnie głębokim zoomie zbioru Mandelbrota [17]. Stosuje arytmetykę arbitralnej precyzji oraz technikę perturbacyjną (ang. *perturbation theory*), która pozwala obliczać wartości iteracji w głębokich powiększeniach bez utraty precyzji wynikającej z ograniczeń arytmetyki zmiennoprzecinkowej. Program jest wysoce wyspecjalizowany — obsługuje niemal wyłącznie zbiór Mandelbrota i pochodne, a jego interfejs jest zoptymalizowany pod kątem generowania głębokich zoomów, nie zaś ogólnej eksploracji czy edukacji.

9.2. Różnice funkcjonalne i jakościowe

Tabela 9.1 zestawia najważniejsze cechy omawianych programów na tle opisywanej aplikacji.

52:2441137599

Tabela 9.1: Porównanie wybranych programów do eksploracji fraktali.

Cecha	FractalSurfer	XaoS	Ultra Fractal	Kalles Fraktaler
Licencja	open source	open source	komercyjna	freeware
Platformy	Win/Mac/Lin	Win/Mac/Lin	Win/Mac	Windows
Język implementacji	Python	C	C++	C++
Przyspieszenie GPU	nie	nie	nie	tak (OpenCL)
Obsługiwane fraktale	5	>15	dowolne	Mandelbrot
Animacja (keyframes)	tak	nie	tak	nie
Eksport wideo	GIF/MP4/WebM	nie	tak	nie
Precyzja obliczeń	float64	float64	arb. precyzja	arb. precyzja
Własne palety	tak	tak	tak	tak

9.3. Wnioski z porównania

Opisywana aplikacja zajmuje niszę między prostymi przeglądarkami fraktali a narzędziami profesjonalnymi. W stosunku do XaoS oferuje moduł animacji z eksportem do popularnych formatów wideo oraz bardziej przejrzysty, ustrukturyzowany interfejs — kosztem niższej wydajności wynikającej z użycia języka Python zamiast C. W stosunku do Ultra Fractal jest rozwiązaniem znacznie prostszym, lecz dostępnym bez opłat licencyjnych i łatwiejszym do uruchomienia bez konfiguracji. Wobec Kalles Fraktaler opisywana aplikacja oferuje szerszy zakres fraktali i animację, lecz ustępuje pod względem możliwości głębokiego zoomu ze względu na ograniczenia arytmetyki 64-bitowej. Kalles Fraktaler wykorzystuje również opcjonalną akcelerację OpenCL dla iteracji perturbacyjnych, co dodatkowo zwiększa jego przewagę wydajnościową przy bardzo głębokich powiększeniach. Akcelerację GPU wymieniono w rozdziale 11 jako jeden z kierunków dalszego rozwoju opisywanej aplikacji.

Kluczową cechą odróżniającą opisywaną aplikację jest jej charakter edukacyjny i implementacyjny: przejrzysta architektura w języku Python, czytelny kod źródłowy oraz spójny potok renderowania sprawiają, że projekt nadaje się nie tylko jako narzędzie eksploracji, lecz także jako punkt wyjścia do dalszego rozwijania lub modyfikowania algorytmów.

Rozdział 10

Dyskusja

Po przedstawieniu wyników naturalnym krokiem jest ich osadzenie w szerszym kontekście. Kolejne podrozdziały zbierają wnioski z bezpośredniej eksploracji fraktali, omawiają rolę kolorowania w odbiorze obiektu matematycznego oraz wskazują ograniczenia przyjętego podejścia implementacyjnego wraz z ich źródłami.

10.1. Wnioski poznawcze

Realizacja projektu pozwoliła zaobserwować kilka nietrywialnych zależności pomiędzy matematyczną definicją fraktala a jego reprezentacją wizualną. Przede wszystkim, nieskończona złożoność granicy zbioru Mandelbrota jest widoczna praktycznie przy każdym poziomie przybliżenia — eksploracja ujawnia coraz drobniejsze struktury, w tym miniatury całego zbioru oraz wzorce powtarzające się z subtelnymi różnicami. Zjawisko to bezpośrednio ilustruje pojęcie samopodobieństwa statystycznego i potwierdza wizualnie własności matematyczne opisane w rozdziale 1.5.

Interaktywna eksploracja pozwala również empirycznie zaobserwować związek między zbiorem Mandelbrota a zbiorami Julii: zmiana parametru c w pobliżu granicy zbioru Mandelbrota prowadzi do dramatycznych zmian kształtu odpowiadającego zbioru fraktala Julii. Zaimplementowany przycisk „go to corresponding Julia set" czyni tę zależność bezpośrednio dostępną dla użytkownika.

10.2. Wpływ kolorowania na percepcję i odbiór fraktali

Sposób kolorowania ma zasadniczy wpływ na to, jakie cechy struktury fraktalnej są widoczne dla obserwatora. Palety o dużym kontraście i nasyceniu (np. *neon*) uwydatniają granice między pasmami iteracyjnymi, natomiast palety o łagodnych przejściach (np. *ice*) eksponują ciągłość gradientu. Korekcja gamma pozwala rozjaśnić ciemne obszary graniczne, ujawniając drobne struktury, które przy domyślnym kolorowaniu byłyby niewidoczne.

Obserwacja ta ma wymiar nie tylko estetyczny, lecz i poznawczy: ten sam obiekt matematyczny, pokolorowany inaczej, może sprawiać wrażenie prostej lub złożonej struktury w zależności od dobranej palety. Podkreśla to rolę wizualizacji jako narzędzia interpretacji, nie tylko prezentacji.

10.3. Ograniczenia przyjętego podejścia

Precyzja numeryczna. Obliczenia prowadzone są w arytmetyce 64-bitowej (`float64`), co ogranicza głębokość dostępnego zoomu. Przy bardzo dużych powiększeniach — przybliżeniu

54:7339004889

przekraczającym około 10^{13} w stosunku do widoku domyślnego, pojawiają się artefakty wynikające z utraty precyzji. Piksele w pobliżu granicy zbioru zaczynają przyjmować niepoprawne wartości, a obraz traci szczegółowość. Przejście na arytmetykę arbitralnej precyzji wymagałoby przebudowy warstwy obliczeniowej i najprawdopodobniej rezygnacji z kompilacji JIT przez Numbę w obecnej postaci.

Obliczenia wyłącznie na CPU. Wszystkie obliczenia wykonywane są przez procesor. Karta graficzna, pomimo swojej znacznie wyższej przepustowości dla równoległych operacji zmiennoprzecinkowych, nie jest wykorzystywana. Renderowanie na GPU (np. przy użyciu shaderów GLSL lub biblioteki CUDA) pozwoliłoby wielokrotnie skrócić czas generowania klatki przy wysokich rozdzielczościach i dużych wartościach `max_iter`.

Jednowątkowe renderowanie. Mimo że aplikacja korzysta z wielu wątków (daemony), sama generacja fraktala odbywa się w jednym wątku roboczym. Numba w trybie `nopython` nie korzysta automatycznie z wielu rdzeni procesora. Równoległe renderowanie wierszy obrazu (ang. *tile rendering*) mogłoby przyspieszyć obliczenia proporcjonalnie do liczby dostępnych rdzeni.

10.4. Potencjalne zastosowania aplikacji

Aplikacja może służyć jako narzędzie dydaktyczne na zajęciach z matematyki, informatyki lub grafiki komputerowej — wizualizując pojęcia iteracji, dynamiki układów i złożoności obliczeniowej w sposób bezpośredni i interaktywny. Możliwość eksportu animacji do formatów MP4 i WebM czyni ją użyteczną również jako generator materiałów wizualnych do prezentacji lub publikacji internetowych.

Rozdział 11

Kierunki rozwoju

Zrealizowana aplikacja stanowi kompletną implementację założonego zakresu, lecz jej architektura była projektowana z myślą o dalszej rozbudowie. W kolejnych podrozdziałach omówiono cztery najistotniejsze kierunki rozwoju, uporządkowane według rosnącej skali wymaganych zmian.

11.1. Dodatkowe fraktale

Naturalnym rozszerzeniem aplikacji byłoby dodanie kolejnych algorytmów fraktalnych. Spośród kandydatów warto wymienić fraktale Newtona — generowane przez iterację metody Newtona dla wielomianów zespolonych, tworzące charakterystyczne wzory z obrotową symetrią — oraz fraktal Lyapunova, oparty na analizie wykładnika Lyapunova dla dwuparametrowej rodziny odwzorowań logistycznych. Obydwa typy dają efekty wizualne jakościowo różne od rodziny z^2+c i wzbogaciłyby zakres tematyczny aplikacji. Warianty takie jak Tricorn („anty-Mandelbrot") czy Burning Ship wyższych rzędów wymagałyby jedynie minimalnych modyfikacji istniejącego kodu escape-time.

11.2. Fraktale trójwymiarowe

Rozszerzenie na trzeci wymiar stanowiłoby znaczący krok rozwojowy. Mandelbulb [18] — trójwymiarowy analog zbioru Mandelbrota, oparty na uogólnieniu potęgowania do przestrzeni $\mathbb{R}^3$ — oraz Mandelbox [19] to obiekty, które od kilkunastu lat cieszą się dużą popularnością w społeczności artystów grafiki generatywnej. Renderowanie takich fraktali wymaga zastosowania techniki śledzenia promieni (ang. *ray marching*) z szacowaniem odległości (ang. *distance estimation*), co znacząco wpłynęłoby na działanie potoku renderowania, różniąc go od tego zaimplementowanego w obecnej wersji. Z uwagi na wysokie wymagania obliczeniowe ta funkcja wymagałaby jednocześnie przejścia na renderowanie z wykorzystaniem GPU.

11.3. Usprawnienia wydajności

Najbardziej impaktowym usprawnieniem byłoby przeniesienie obliczeń na kartę graficzną za pomocą shaderów (np. GLSL przez bibliotekę `moderngl`) lub interfejsu CUDA/OpenCL. GPU oferuje tysiące równoległych jednostek obliczeniowych idealnie dopasowanych do charakteru algorytmów escape-time, w których każdy piksel jest niezależny. Alternatywnie, w ramach platformy CPU, możliwe jest zastosowanie wielowątkowego renderowania kafelkowego z dekoratorem `@jit(parallel=True)` biblioteki Numba oraz dyrektywą `prange` zamiast zwykłej pętli `for`.

56:2876567554

Osobnym kierunkiem jest implementacja arytmetyki arbitralnej precyzji dla fraktali klasy escape-time, co zniosłoby obecne ograniczenie głębokości zoomu wynikające z użycia `float64`.

11.4. Wariant edukacyjny

Aplikacja mogłaby zostać rozbudowana o tryb edukacyjny, w którym obok obrazu fraktala wyświetlane są dodatkowe informacje: aktualne równanie iteracji, wartość z_n dla wskazanego piksela śledzona krok po kroku, wizualizacja orbity konkretnego punktu na płaszczyźnie zespolonej oraz odnośniki do pojęć matematycznych. Taki tryb uczyniłby program użytecznym narzędziem dydaktycznym, pozwalającym obserwować działanie algorytmu nie tylko jako efekt wizualny, lecz jako konkretny proces obliczeniowy.

Rozdział 12

Podsumowanie

Rozdział zamyka pracę zestawieniem zrealizowanych funkcjonalności w odniesieniu do wymagań z rozdziału 3, oceną trafności najważniejszych decyzji projektowych w świetle uzyskanych wyników oraz refleksją nad umiejętnościami rozwiniętymi w trakcie realizacji projektu.

12.1. Zrealizowane funkcje

Aplikacja FractalSurfer w obecnej wersji realizuje wszystkie wymagania funkcjonalne i niefunkcjonalne sformułowane w rozdziale 3, w tym te oznaczone jako opcjonalne.

Zrealizowane wymagania funkcjonalne.
— Zaimplementowano pięć algorytmów fraktalnych: zbiór Mandelbrota, zbiór Julii, Burning Ship, trójkąt Sierpińskiego oraz paproć Barnsleya. Każdy z nich operuje w ramach jednolitego potoku renderowania opartego o klasę bazową `FractalAlgorithm` i dostarcza własny zestaw parametrów dziedziczący po `ReactiveParams`.
— Nawigację kamerą za pomocą klawiatury (`W/A/S/D` — przesunięcie, `Q/E` — przybliżenie i oddalenie) z modyfikatorami przyspieszającymi i spowalniającymi ruch.
— System reaktywnych parametrów, w którym każda zmiana wartości suwaka automatycznie wyzwala przerysowanie obrazu.
— Konfigurowalne kolorowanie obejmujące cztery presety (*classic*, *fire*, *ice*, *neon*) oraz tryb niestandardowy ze swobodnie definiowanymi punktami kontrolnymi gradientu i regulowaną korekcją gamma.
— Zapis aktualnego widoku do pliku graficznego PNG.
— Dynamiczne dostosowanie układu interfejsu do bieżącego rozmiaru okna aplikacji.
— Moduł animacji oparty o klatki kluczowe z dwoma trybami interpolacji ruchu kamery (liniowym i krzywą Catmull-Rom).
— Eksport animacji do trzech formatów wyjściowych: GIF, MP4 i WebM.

Zrealizowane wymagania niefunkcjonalne.
— Responsywność interfejsu zapewniona poprzez przeniesienie obliczeń do dedykowanego wątku demonicznego (`RedrawDaemon`) oraz obsługę klawiatury w drugim wątku (`KeyboardDaemon`).
— Adaptacyjne obniżanie jakości renderowania podczas aktywnego ruchu kamery, opisane w podrozdziale 8.4.2, pozwalające utrzymać płynną nawigację nawet w wymagających ustawieniach.
— Obsługa rozdzielczości wyjściowej co najmniej 1200×800 pikseli ze swobodnym powiększaniem okna.
— Przenośność między systemami Windows, macOS i Linux potwierdzona testami opisanymi w podrozdziale 8.3.1.

58:1912115827

— Deterministyczność eksportu zapewniona przez ustawiane ziarno generatora liczb pseudolosowych dla fraktali IFS oraz pełną deterministyczność algorytmów escape-time wynikającą ze standaryzowanej arytmetyki IEEE 754.

12.2. Rezultaty i wnioski

Realizacja projektu zweryfikowała trafność przyjętych założeń projektowych oraz zidentyfikowała mocne i słabe strony zastosowanych rozwiązań. Test punktów charakterystycznych przedstawiony w podrozdziale 8.1 potwierdził poprawne klasyfikowanie wszystkich rozstrzygalnych punktów zbioru Mandelbrota, a weryfikacja krzyżowa z istniejącymi narzędziami (XaoS, Kalles Fraktaler) nie ujawniła rozbieżności strukturalnych. Pomiary przedstawione w podrozdziale 8.3 potwierdziły teoretyczne złożoności obliczeniowe algorytmów — $O(W \cdot H \cdot N_{\max})$ dla fraktali escape-time oraz $O(N)$ dla fraktali IFS — a w typowych warunkach użycia (rozdzielczość 1200×800, $N_{\max} = 100$–300) czas generowania pojedynczej klatki na sprzęcie konsumenckim nie przekracza 100–150 ms, co pozwala na płynną nawigację w czasie rzeczywistym; dla bardziej wymagających ustawień kluczowy jest mechanizm adaptacyjnego obniżania jakości, który pozwala zachować responsywność kosztem tymczasowego spadku rozdzielczości i liczby iteracji.

Trafnym założeniem projektowym okazało się rygorystyczne oddzielenie warstwy logicznej od interfejsu użytkownika, w połączeniu z systemem reaktywnych parametrów i jednolitą klasą bazową `FractalAlgorithm` — dodanie kolejnego algorytmu fraktalnego sprowadza się do utworzenia jednej klasy implementacyjnej, jednego zestawu parametrów oraz dodania wartości do enum `AlgoChoice`, bez modyfikacji istniejącego kodu. Decyzja o wykorzystaniu Numby dla wewnętrznych pętli obliczeniowych pozwoliła osiągnąć wydajność porównywalną z implementacją w językach kompilowanych przy zachowaniu zwięzłości i czytelności kodu Pythona. Najbardziej zauważalnym ograniczeniem przyjętego podejścia jest precyzja arytmetyki `float64`, ograniczająca głębokość zoomu do współczynnika rzędu 10^{12}–10^{13} (podrozdział 8.4); drugim istotnym ograniczeniem jest brak akceleracji GPU — wszystkie obliczenia wykonywane są przez CPU, co dla bardzo wysokich rozdzielczości lub dużych wartości $N_{\max}$ przekłada się na czas renderowania mierzony w setkach milisekund lub w sekundach. Oba ograniczenia są naturalnym kierunkiem dalszego rozwoju, opisanym w rozdziale 11.

Cel pracy — zaprojektowanie i implementacja aplikacji umożliwiającej interaktywną eksplorację fraktali z zachowaniem płynności sterowania oraz jakości wizualizacji — został zrealizowany. Aplikacja stanowi narzędzie łączące walory edukacyjne, eksploracyjne i artystyczne, mieszcząc się w niszy między prostymi przeglądarkami fraktali a profesjonalnymi pakietami komercyjnymi, a otwarta architektura kodu i przejrzysta struktura modułów czynią projekt punktem wyjścia do dalszych modyfikacji oraz rozszerzeń, których kierunki nakreślono w rozdziale 11.

12.3. Rozwój umiejętności i doświadczeń

Realizacja niniejszej pracy umożliwiła rozwój umiejętności w trzech głównych obszarach: matematyki stosowanej, inżynierii oprogramowania oraz projektowania systemów interaktywnych.

Matematyka. Centralnym elementem pracy była praktyczna implementacja zagadnień z dynamiki układów dyskretnych w przestrzeni zespolonej. Konieczność sprowadzenia formalnych

59:8633064227

definicji zbiorów Mandelbrota i Julii do postaci wykonalnej numerycznie wymagała pełnego zrozumienia kryteriów ucieczki, struktury orbit, a także związków pomiędzy parametrem c a własnościami otrzymanego zbioru. Implementacja fraktali punktowych — trójkąta Sierpińskiego i paproci Barnsleya — pozwoliła z kolei zaznajomić się z teorią iterowanych układów funkcji, w tym pojęciami przekształcenia zwężającego, atraktora oraz rozkładu prawdopodobieństw przekształceń afinicznych.

Inżynieria oprogramowania. Projekt wymagał integracji kilku zaawansowanych technik programistycznych w spójną całość. Najistotniejszymi z nich były: zaprojektowanie wielowątkowego interfejsu graficznego z dedykowanymi wątkami demonicznymi do renderowania i obsługi klawiatury; zintegrowanie kompilacji JIT z biblioteki Numba z ekosystemem NumPy przy zachowaniu jednolitego typu opakowującego `Double` we wszystkich obliczeniach; podział kodu na głęboko rozdzielone warstwy (`common`, `fractal`, `ui`) z jednoznacznymi zależnościami między nimi. Konsekwentne stosowanie wzorca reaktywnych parametrów (`ReactiveParams`) okazało się szczególnie wartościową lekcją — pokazało ono jak prosty mechanizm obserwowanych atrybutów może wyeliminować znaczną część kodu typu „glue" łączącego widżety z logiką aplikacji. Taka separacja okazała się również nieoceniona przy testowaniu i iteracyjnym rozwijaniu kolejnych komponentów.

Projektowanie kompromisów. Najtrudniejszym, lecz zarazem najbardziej kształcącym aspektem pracy okazało się znalezienie równowagi pomiędzy jakością generowanego obrazu a responsywnością interfejsu. Wymagało to zarówno decyzji algorytmicznych (mechanizm adaptacyjnego obniżania jakości, memoizacja chmury punktów dla fraktali IFS), jak i architektonicznych (separacja wątku obliczeniowego od głównej pętli zdarzeń, użycie `threading.Event` zamiast aktywnego oczekiwania). Doświadczenie to ukazało, że projektowanie aplikacji interaktywnych nie sprowadza się do optymalizacji wyłącznie jednej miary — właściwym celem jest staranne wyważenie wielu czynników współzależnych w odpowiedzi na postawione wymagania funkcjonalne.

Słownik

W rozdziale zebrano pojęcia specjalistyczne pojawiające się w pracy wraz z ich definicjami. Hasła uporządkowano alfabetycznie; anglojęzyczne odpowiedniki podano w nawiasach.

Adaptacyjne obniżanie jakości (*downscaling*) : Tymczasowe zmniejszenie rozdzielczości i liczby iteracji podczas ruchu kamery w celu utrzymania płynności interfejsu.

Arytmetyka arbitralnej precyzji : Reprezentacja liczb o liczbie cyfr znaczących dobieranej dynamicznie, nieograniczonej formatem sprzętowym.

Atraktor : Zwarty zbiór, do którego zbiegają trajektorie układu dynamicznego z jego basenu przyciągania.

Burning Ship : Fraktal escape-time oparty na iteracji $z_{n+1} = (|\mathrm{Re}(z_n)| + i|\mathrm{Im}(z_n)|)^2 + c$.

Chaos game : Metoda przybliżania atraktora IFS przez losowe stosowanie kolejnych przekształceń składowych do bieżącego punktu.

DearPyGui : Biblioteka Pythona do tworzenia interfejsów graficznych z akceleracją GPU, oparta na Dear ImGui.

Float64 (IEEE 754, podwójna precyzja) : 64-bitowa liczba zmiennoprzecinkowa o około 15–17 cyfrach znaczących mantysy.

Fraktal : Obiekt geometryczny o złożonej strukturze zachowującej cechy samopodobieństwa na różnych skalach.

Fraktal escape-time : Fraktal generowany przez przypisanie każdemu pikselowi liczby iteracji do potrzebnej do przekroczenia progu ucieczki.

Fraktal punktowy : Fraktal generowany jako chmura punktów wyznaczanych iteracyjnie przez układ IFS i rasteryzowanych do siatki pikseli; podejście alternatywne do fraktali escape-time.

Gradient barwny : Ciągłe odwzorowanie wartości $t \in [0, 1]$ na barwę RGB, definiowane przez listę punktów kontrolnych.

IFS (*Iterated Function System*) : Skończona rodzina przekształceń zwężających, których iteracyjne stosowanie wyznacza atraktor o strukturze fraktalnej.

Iteracja : Wielokrotne stosowanie tej samej funkcji: $z_{n+1} = f(z_n)$.

JIT (*Just-In-Time compilation*) : Kompilacja kodu do postaci natywnej w czasie pierwszego wywołania funkcji.

Kamera (kadr) : Obiekt opisujący stan widoku przez przesunięcie środka kadru i skalę obserwacji.

Klatka kluczowa (*keyframe*) : Zapamiętany stan kamery używany jako punkt kontrolny przy interpolacji animacji.

Korekcja gamma : Nieliniowa transformacja jasności postaci $\mathbf{c} \mapsto \mathbf{c}^{1/\gamma}$.

Kryterium ucieczki : Warunek $|z_n| > 2$ rozstrzygający o rozbieżności orbity dla rodziny $z^2 + c$.

Krzywa Catmull-Rom : Interpolujący splajn sześcienny przechodzący dokładnie przez punkty kontrolne z zachowaniem ciągłości pierwszej pochodnej.

61:3338156081

Liczby zespolone : Liczby postaci $z = a + bi$, gdzie $a, b \in \mathbb{R}$ oraz $i^2 = -1$.

LUT (*look-up table*) : Tablica wstępnie obliczonych wartości używana do szybkiego mapowania danych bez powtarzania obliczeń.

Memoizacja : Zapamiętywanie wyników wywołań funkcji w celu uniknięcia powtórnych obliczeń przy tych samych argumentach.

Numba : Biblioteka realizująca kompilację JIT kodu Pythona z wykorzystaniem infrastruktury LLVM.

NumPy : Biblioteka Pythona do wydajnych operacji na wielowymiarowych tablicach liczbowych.

Orbita : Ciąg wartości $\{z_n\}_{n \geqslant 0}$ generowany przez iterację funkcji od punktu startowego z_0.

Paproć Barnsleya : Atraktor IFS złożonego z czterech przekształceń afinicznych, przypominający kształtem liść paproci.

Pasmo iteracyjne : Widoczny obszar w obrazie odpowiadający pikselom o tej samej całkowitoliczbowej liczbie iteracji.

Polling stagger : Minimalny odstęp czasowy między kolejnymi krokami pętli aktualizującej stan kamery.

Próg ucieczki : Wartość, której przekroczenie przez moduł $|z_n|$ traktowane jest jako rozbieżność orbity.

Przekształcenie afiniczne : Złożenie przekształcenia liniowego z translacją: $T(\mathbf{x}) = A\mathbf{x} + \mathbf{b}$.

Punkt kontrolny gradientu (*colour stop*) : Para $(t, \mathbf{c})$ wiążąca pozycję $t \in [0, 1]$ z barwą RGB.

Punkt stały : Punkt x^* funkcji f spełniający $f(x^*) = x^*$.

Rasteryzacja : Przekształcenie chmury punktów na obraz złożony z dyskretnej siatki pikseli.

Reaktywny parametr : Parametr, którego zmiana automatycznie wyzwala odpowiednie operacje bez ręcznego wiązania zdarzeń.

Samopodobieństwo : Własność polegająca na strukturalnym podobieństwie fragmentów obiektu do całości po przeskalowaniu.

Smooth coloring (*normalized iteration count*) : Technika eliminacji pasm iteracyjnych przez zastąpienie liczby iteracji wartością ciągłą $\mu = n - \log(\log r)/\log 2$.

Splajn : Funkcja kawałkami wielomianowa o określonej gładkości w punktach łączenia segmentów.

Tekstura dynamiczna : Tekstura GPU, której zawartość pikseli może być wielokrotnie aktualizowana w trakcie działania programu.

Trójkąt Sierpińskiego : Klasyczny fraktal o wymiarze pudełkowym $D_B = \log 3/\log 2 \approx 1{,}585$.

Wątek demoniczny (*daemon thread*) : Wątek tła kończony automatycznie wraz z zakończeniem programu głównego.

Wektoryzacja : Wykonywanie operacji na całych tablicach jako pojedynczej operacji skompilowanej zamiast pętli interpretowanej.

Wieloskalowość : Brak charakterystycznej skali długości; każdy poziom przybliżenia ujawnia nowe szczegóły.

Wymiar fraktalny (*box-counting dimension*) : Miara wymiaru zbioru, mogąca przyjmować wartości niecałkowite, zdefiniowana jako $D_B = \lim_{\varepsilon \to 0} \log N(\varepsilon)/\log(1/\varepsilon)$.

Zbiór Julii : Zbiór punktów startowych z_0, dla których orbita pod działaniem $f_c(z) = z^2 + c$ pozostaje ograniczona, dla ustalonego c.

Zbiór Mandelbrota : Zbiór parametrów $c \in \mathbb{C}$, dla których orbita punktu $z_0 = 0$ pod działaniem $f_c(z) = z^2 + c$ pozostaje ograniczona.

62:5947208792

Ziarno generatora (*seed*) : Wartość inicjująca generator liczb pseudolosowych, zapewniająca powtarzalność wyników.

Zoom (skala kamery) : Stosunek odległości w przestrzeni fraktala do odległości w pikselach.

Bibliografia

[1] Edwin Catmull, Raphael Rom, *A Class of Local Interpolating Splines*, Computer Aided Geometric Design, Academic Press, 1974.

[2] Benoit B. Mandelbrot, *The Fractal Geometry of Nature*, W.H. Freeman and Company, Nowy Jork, 1982.

[3] Michael F. Barnsley, *Fractals Everywhere*, Academic Press, Boston, 1988.

[4] Heinz-Otto Peitgen, Peter H. Richter, *The Beauty of Fractals: Images of Complex Dynamical Systems*, Springer-Verlag, Berlin, 1986.

[5] Kenneth Falconer, *Fractal Geometry: Mathematical Foundations and Applications*, John Wiley & Sons, Chichester, 1990.

[6] Michael Michelitsch, Otto E. Rössler, *The „burning ship" and its quasi-Julia sets*, Computers & Graphics 16(4), 1992.

[7] Heinz-Otto Peitgen, Hartmut Jürgens, Dietmar Saupe, *Chaos and Fractals: New Frontiers of Science*, Springer-Verlag, Nowy Jork, 1992.

[8] F. Garcia, A. Fernandez, J. Barrallo, L. Martin, *Coloring Dynamical Systems in the Complex Plane*, Generative Art Conference, Mediolan, 2002.

[9] Python Software Foundation, *Python 3.12 Documentation*, 2023. `https://docs.python.org/3.12/` [dostęp: maj 2026].

[10] Charles R. Harris et al., *Array programming with NumPy*, Nature, t. 585, s. 357–362, 2020. `https://doi.org/10.1038/s41586-020-2649-2`

[11] Siu Kwan Lam, Antoine Pitrou, Stanley Seibert, *Numba: A LLVM-based Python JIT Compiler*, Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC, 2015. `https://doi.org/10.1145/2833157.2833162`

[12] Jonathan Hoffstadt et al., *DearPyGui: A Modern, Fast and Powerful GUI Framework for Python*, repozytorium GitHub, 2021. `https://github.com/hoffstadt/DearPyGui` [dostęp: maj 2026].

[13] Alex Clark et al., *Pillow (PIL Fork) Documentation*, 2023. `https://pillow.readthedocs.io/` [dostęp: maj 2026].

[14] imageio contributors, *imageio Documentation*, 2023. `https://imageio.readthedocs.io/` [dostęp: maj 2026].

[15] XaoS Project, *XaoS — Real-time Interactive Fractal Zoomer*, `https://xaos-project.github.io/` [dostęp: maj 2026].

[16] Frederik Slijkerman, *Ultra Fractal — Advanced Fractal Software*, `https://www.ultrafractal.com/` [dostęp: maj 2026].

[17] Karl Runmo, *Kalles Fraktaler*, `https://mathr.co.uk/kf/` [dostęp: maj 2026].

[18] Daniel White, *Mandelbulb*, `https://www.skytopia.com/project/fractal/2mandelbulb.html` [dostęp: maj 2026].

[19] Tom Lowe, *Mandelbox*, `https://sites.google.com/site/mandelbox/` [dostęp: maj 2026].

64:4235051369

Dodatek

Dodatek zawiera materiały uzupełniające, niewchodzące w zakres głównej części pracy: instrukcję użytkownika obejmującą wymagania środowiskowe, procedurę instalacji oraz opis funkcji i sterowania aplikacją FractalSurfer.

12.4. Wymagania i instalacja

Aplikacja wymaga 64-bitowego systemu operacyjnego oraz interpretera Python w odpowiedniej wersji. Do zarządzania środowiskiem i zależnościami wykorzystywane jest narzędzie `uv`.

Wymagania środowiskowe.
— System operacyjny 64-bitowy; Windows 10 lub nowszy, macOS 26 lub nowszy, dowolna współczesna dystrybucja Linux.
— Interpreter Python w wersji co najmniej 3.12. Wersje 3.14 i nowsze nie są wspierane ze względu na ograniczenia kompatybilności biblioteki Numba.

Aplikacja nie wymaga połączenia z internetem (po wykonaniu instalacji) ani uprawnień administratora — wszystkie zależności są pobierane do izolowanego środowiska wirtualnego w katalogu projektu.

Instalacja projektu. Projekt korzysta z narzędzia `uv` do zarządzania środowiskiem Pythona oraz jego zależnościami. Procedura instalacji obejmuje trzy kroki:
1. Zainstalowanie `uv` zgodnie z instrukcjami na stronie `https://docs.astral.sh/uv/`.
2. Pobranie kodu źródłowego projektu i przejście do jego katalogu głównego.
3. Wywołanie polecenia `uv sync` w terminalu, które automatycznie pobierze wymagane zależności i utworzy środowisko wirtualne.

Po pomyślnym wykonaniu powyższych kroków aplikacja jest gotowa do uruchomienia.

12.5. Uruchomienie programu

Uruchomienie aplikacji odbywa się poprzez wywołanie polecenia `uv run gui` w katalogu głównym projektu. Narzędzie `uv` aktywuje środowisko wirtualne utworzone podczas instalacji i uruchamia interfejs użytkownika.

Po uruchomieniu otwiera się okno główne, z domyślnie wybranym widokiem zbioru Mandelbrota. Pierwsze przełączenie się na zakładkę nieskompilowanego jeszcze algorytmu skutkuje jednorazowym wykonaniem kompilacji JIT biblioteki Numba, co może objawić się jako krótkotrwałe „zacięcie” interfejsu graficznego (zob. podrozdział 8.3) — kolejne przełączenia na ten sam typ algorytmu są natychmiastowe.

65:5796773400

12.6. Opis funkcji i sterowania

Interfejs aplikacji składa się z dwóch podstawowych obszarów: panelu wyświetlającego obraz po lewej stronie ekranu, oraz panelu sterowania po prawej. Panel boczny podzielony jest na sekcje, które zostały opisane poniżej.

Aby rozpocząć eksplorację domyślnie wyświetlanego fraktala Mandelbrota, należy użyć klawiatury do poruszania kamerą.

Sterowanie kamerą.
— `W` / `A` / `S` / `D` — przesunięcie widoku w kierunkach kardynalnych.
— `Q` / `E` — przybliżenie i oddalenie widoku.
— `Shift` / `Ctrl` — przyspieszenie i spowolnienie ruchu (domyślne mnożniki prędkości: ×1,5 oraz ×0,5). Jednoczesne użycie obu modyfikatorów skutkuje zniesieniem ich efektów; obowiązuje prędkość bazowa.
Modyfikatory `Shift` oraz `Ctrl` można dowolnie łączyć z klawiszami przesunięcia oraz zoomu.

Wybór algorytmu. Górna część panelu bocznego zawiera pasek zakładek z pięcioma wyróżnionymi algorytmami: *Mandelbrot*, *Julia*, *Fern*, *Sierpinski*, oraz *Burning Ship*. Każda zakładka wyświetla specyficzne dla algorytmu widżety do manipulacji parametrami algorytmów: (np. `max iter` dla fraktali klasy escape-time, `seed` i `num points` dla fraktali IFS). Zakładka *Mandelbrot* zawiera dodatkowo przycisk `go to corresponding Julia set`, który odczytuje aktualne współrzędne środka kadru i przekazuje je jako parametr c do zakładki *Julia*, automatycznie ją aktywując.

Sekcja *camera settings*. Zwijalna sekcja zawierająca:
— przycisk `reset camera`, przywracający widok domyślny aktualnie wybranego algorytmu,
— przycisk `save screenshot` (opisany szczegółowo w podrozdziale 12.6.1),
— etykietę z bieżącymi koordynatami środka kadru,
— pole wyboru `movement downscaling` oraz suwaki konfigurujące mechanizm adaptacyjnego obniżania jakości obrazu (`downscale factor`, `iteration limit`),
— suwaki sterowania nawigacją: `movement speed`, `zoom speed`, `Shift speed modifier`, `Ctrl speed modifier`, oraz `polling stagger` (minimalny odstęp czasowy między kolejnymi odświeżeniami widoku kamery).

Sekcja *animations*. Sterowanie modułem animacji:
— `animation mode` — wybór trybu interpolacji (*linear* lub *spline*),
— `export format` — wybór formatu wyjściowego (GIF, MP4, WebM),
— `frame rate` — liczba klatek przypadająca na pojedynczy segment animacji (zakres 1–60),
— `add keyframe` — dodaje aktualne parametry kamery jako nową klatkę kluczową (etykieta przycisku zawiera aktualną liczbę klatek),
— `clear keyframes` — usuwa wszystkie uprzednio zarejestrowane klatki kluczowe,
— `export animation` — generuje i zapisuje animację klatek kluczowych na podstawie wybranych parametrów.

Sekcja *colours*. Konfiguracja kolorowania:
— suwak `gamma` — korekcja gamma w zakresie 0,01–1,00 (wartość 1,00 oznacza brak korekcji),
— `palette mode` — wybór trybu palety (*preset* lub *custom*),

66:1093822628

— w trybie `preset` dostępny jest wybór jednej z czterech predefiniowanych palet chromatycznych (*classic*, *fire*, *ice*, *neon*),
— w trybie `custom` pojawia się edytor punktów kontrolnych gradientów z przyciskiem `add stop` dodającym nowe punkty, oraz przyciskiem `delete` przy każdym punkcie; każdy punkt składa się z suwaka punktu odcięcia $t \in [0, 1]$ oraz selektora koloru w formacie RGB.

12.6.1. Eksport i przykładowy workflow

Zrzuty ekranu. Przycisk `save screenshot` w sekcji *camera settings* zapisuje aktualną zawartość panelu obrazu do pliku PNG w katalogu `screenshots/`, położonym w katalogu roboczym aplikacji. Nazwa pliku zawiera znacznik czasu w formacie `YYYY-MM-DD_HH-MM-SS.png`. Po pomyślnym zapisie obok przycisku wyświetlany jest komunikat `saved` *`nazwa_pliku`*.

Workflow tworzenia animacji. Typowy proces tworzenia animacji obejmuje następujące kroki:
1. Nawigacja klawiszami `WASD`/`QE` do pierwszej interesującej pozycji kamery.
2. Naciśnięcie przycisku `add keyframe` — jego etykieta aktualizuje się do `add keyframe (1 total)`.
3. Nawigacja do kolejnej pozycji i ponowne wciśnięcie `add keyframe`; krok powtarzany jest dla każdej pożądanej klatki kluczowej.
4. Wybór trybu interpolacji (*linear*/*spline*) oraz formatu wyjściowego (GIF, MP4, WebM).
5. Ustawienie częstotliwości klatek (`frame rate`) — każdy segment między sąsiednimi klatkami kluczowymi zawiera dokładnie `frame rate` klatek interpolowanych, co przy odtwarzaniu z tą samą częstotliwością daje stałą długość segmentu wynoszącą jedną sekundę; parametr kontroluje zatem płynność, lecz nie długość animacji.

Pliki eksportowane są do podkatalogów `animations/gif/`, `animations/mp4/` lub `animations/webm/` w katalogu roboczym aplikacji, w zależności od wybranego formatu. Nazwa pliku zawiera znacznik czasu w formacie analogicznym do tego używanego przez zrzuty ekranu.

Czas eksportu. W przeciwieństwie do nawigacji interaktywnej, eksport animacji zawsze odbywa się w pełnej jakości, bez stosowania mechanizmu adaptacyjnego obniżania rozdzielczości. Każda klatka generowana jest przy aktualnie ustawionych parametrach algorytmu i rozdzielczości okna, dlatego całkowity czas dokonania eksportu może przekroczyć kilkadziesiąt sekund. W tym czasie interfejs programu pozostaje niereaktywny. Stan operacji sygnalizowany jest etykietą `working...`, która zmienia się na `saved` *`nazwa_pliku`* przy ukończeniu procesu.

Podziękowanie

Pragnę serdecznie podziękować Promotorowi, dr inż. Bartoszowi Nowakowi, za życzliwą opiekę merytoryczną, cenne uwagi oraz cierpliwość okazaną w trakcie realizacji niniejszej pracy. Szczególne podziękowania kieruję do moich rodziców — za wiarę we mnie i wsparcie, które towarzyszyło mi przez wszystkie lata edukacji. Dziękuję również Patrykowi Jakubczykowi i Wojciechowi Jarząbkowi za rozmowy, motywację i pomoc, na którą zawsze mogłem liczyć.

68:6968056840

Streszczenie

W pracy przedstawiono projekt i implementację aplikacji służącej do interaktywnej eksploracji fraktali oraz generowania wizualizacji o walorach edukacyjnych i artystycznych. Zidentyfikowano dwa kluczowe aspekty problemu: wysoką złożoność obliczeniową generowania fraktali oraz konieczność zapewnienia użytkownikowi płynnej interakcji (nawigacji, zmiany parametrów, doboru kolorowania) przy jednoczesnym zachowaniu jakości obrazu.

W ramach pracy opracowano zestaw algorytmów obejmujący fraktale escape-time (zbiór Mandelbrota, zbiór Julii, Burning Ship) oraz fraktale punktowe generowane metodami iteracyjnymi (trójkąt Sierpińskiego, paproć Barnsleya). Zaimplementowano potok renderowania oparty o tablicę wartości iteracyjnych i mapowanie kolorów z wykorzystaniem tablic LUT i gradientów barwnych, w tym tryb presetów oraz tryb definiowania własnej palety.

Kluczowym elementem rozwiązania jest mechanizm adaptacyjnego obniżania jakości renderowania podczas aktywnej nawigacji kamerą — pozwala on utrzymać responsywność interfejsu kosztem tymczasowego spadku rozdzielczości i liczby iteracji, a po zakończeniu ruchu obraz jest natychmiast renderowany w pełnej jakości. Drugim wyróżniającym elementem jest moduł animacji oparty o klatki kluczowe i interpolację ruchu kamery (liniową lub krzywą Catmull-Rom), z eksportem do formatów GIF, MP4 i WebM.

Aplikacja umożliwia również sterowanie kamerą za pomocą klawiatury oraz zapis zrzutów ekranu. W celu poprawy wydajności zastosowano kompilację JIT (Numba) oraz memoizację wyników dla fraktali punktowych. Architektura kodu opiera się na ścisłej separacji warstwy logicznej od warstwy interfejsu użytkownika, co umożliwia niezależny rozwój i testowanie poszczególnych komponentów.

69:9168914361

Abstract

Interactive, Impressive Fractal Exploration Program

This thesis presents the design and implementation of an application for the interactive exploration of fractals and the generation of visualizations with both educational and artistic value. Two key aspects of the problem were identified: the high computational cost of fractal generation and the necessity of providing the user with smooth interaction (navigation, parameter changes, colour selection) while preserving image quality.

The work covers a set of algorithms including escape-time fractals (the Mandelbrot set, Julia sets, Burning Ship) and point-based fractals generated by iterative methods (the Sierpinski triangle, Barnsley's fern). A rendering pipeline is implemented around an iteration-value lattice and a colour mapping based on look-up tables and colour gradients, including a preset mode and a custom-palette mode.

A central element of the solution is an adaptive quality-reduction mechanism active during camera navigation: it preserves UI responsiveness at the cost of a temporary drop in resolution and iteration count, and renders the image at full quality immediately once movement ends. A second distinguishing element is an animation module based on keyframes and camera-motion interpolation (linear or Catmull-Rom spline), with export to GIF, MP4 and WebM formats.

The application additionally supports keyboard-based camera control and screenshot saving. For performance, the implementation uses JIT compilation (Numba) and memoization of point-based fractal results. The code architecture is based on a strict separation between the logic layer and the user-interface layer, enabling independent development and testing of individual components.

70:8257867796